RESERVIFY® Implementation Phase

Milestone 4:

Component 1(Final Project Submission) +

Component 2(Demonstration)

Group Number: 32

Team Name: TechFusion

TechFusion Partner Details

Member:	Name:	Surname:	Student Number:
1	Arno	Strauss	2613224
2	Dintle	Maine	2451837
3	Khulekani	Mtshali	2550411

TechFusion Logo

TABLE OF CONTENTS

1. INTRODUCING RESERVIFY

DESCRIPTION

Reservify: A University Venue Booking System

Reservify is an innovative platform designed to revolutionize the venue booking process at the University of the Witwatersrand. By automating key tasks such as venue selection, booking confirmation, and capacity management, Reservify replaces the outdated and error-prone manual system. This centralized platform offers a user-friendly interface, making it easy for students, lecturers, and administrators to efficiently find, reserve, and manage university spaces for a variety of events, including lectures, seminars, exams, meetings, and more.

Reasons for Creating Reservify

The current venue booking process at WITS University is plagued by inefficiencies, characterized by a manual, fragmented, and time-consuming approach. The absence of an automated solution leads to significant resource wastage. Key issues include:

Scheduling Conflicts	The absence of a centralized system results in frequent double bookings and scheduling conflicts, causing frustration and wasted time for users trying to secure venues.
Communication Gaps	Ineffective communication between users and administrators often results in missed bookings and unattended requests. This not only strains university resources but also diminishes overall user satisfaction.
Resource Wastage	The inefficiencies inherent in the current manual system led to the underutilization of valuable university resources. Excessive hours are spent by staff managing bookings, diverting their focus from more strategic initiatives.

In the university setting, students and lecturers frequently need to book venues for a variety of purposes—ranging from lectures and conferences to society meetings. However, the **existing process has significant limitations**:

Limited Booking Options	Students currently lack the ability to book venues for society meetings.
Ineffective Capacity Management	The system struggles to efficiently manage venue capacity, leading to mismatched bookings.
Reliance on Manual Processes	Administrators currently manage all bookings, which often leads to backlogs and confusion, further exacerbating resource wastage. The dependence on manual bookings by school administrators or the facilities department is both time-consuming and error-prone, resulting in scheduling conflicts and difficulties in finding suitable venues.

Reservify is designed to tackle these challenges by offering a categorized venue selection system. This feature allows users—students, lecturers, and administrators—to easily find, book and manage spaces that align with their specific event requirements.

The platform not only facilitates seamless bookings but also enables administrators to efficiently oversee and manage the entire process. By addressing these issues, **Reservify will enhance accessibility, boost efficiency, and improve the overall user experience** for all stakeholders involved in venue bookings within the university environment.

Furthermore, Reservify is built to accommodate the complexities of an ever-growing academic landscape. By significantly enhancing operational efficiency and simplifying the management of campus venues, the platform saves time and fosters better coordination across the university community.

GOALS	DESCRIPTION
Efficiency enhancement	Implement automated processes and validations to **reduce manual effort and errors**, thereby saving time and resources in the venue booking workflow.
Capacity Management	Develop a robust system that intelligently allocates venues based on event capacity, **preventing overbooking**, and optimizing resource utilization.
Diverse booking options	Provide a **comprehensive range of venue categories** to cater to various events such as lectures, society meetings, group meetings, staff meeting, conferences and presentations, empowering users with suitable choices.
User Empowerment	Empower users with **self-service capabilities** to manage their booking details, update event information, and make necessary adjustments, reducing dependency on administrative support for routine tasks.

Key Requirements of Reservify

Requirement Type	Description
Functional Requirements	Reservify provides features like user registration, role-based access (students, lecturers, administrators) and venue availability checks. The system allows users to book venues according to event type, capacity, and duration, while administrators can monitor all activities and generate booking reports.
Non-Functional Requirements	The system ensures scalability to handle growing users, performance to provide quick responses, and security to protect user data. It is designed for 24/7 availability with minimal downtime and follows standard coding practices to allow easy maintenance.

Functional Requirements for Reservify

Reservify aims to provide an effective solution by delivering essential functions that meet user needs while ensuring smooth operations for all involved parties.

Requirement	Description	Impact on User Bases
User Registration and Login	Students and lecturers must register by providing personal details like name, email, and contact number. Administrators do not register but log in with secure credentials.	**Students & Lecturers**: Ensures that personal information is stored for future bookings. **Administrators**: Manage users and bookings without registration, providing them with full system control.
Venue Selection	Users can select venues based on categories (lecture rooms, conference rooms), event type, capacity, and dates.	**Students & Lecturers**: They can filter venues that fit their needs easily. **Administrators**: Oversee all venue selections and ensure bookings follow policies.
Venue Availability Checks & Notifications	The system checks venue availability and suggests alternatives if a venue is unavailable.	**Students & Lecturers**: They are informed in real-time about availability, avoiding conflicts. **Administrators**: Can adjust or override bookings for priority events.
View, Update, or Cancel Bookings	Users can manage their bookings, including modifying or cancelling them.	**Students & Lecturers**: Offers flexibility to change bookings when needed. **Administrators**: Have full control to edit or cancel any booking for resource management.
Administrator Functions	Administrators can manage bookings, users, and generate detailed reports.	**Students & Lecturers**: Benefit from a well-managed system free from errors. **Administrators**: Use reports to make data-driven decisions and ensure smooth operations.
Booking Validation	The system validates booking inputs such as capacity, date, and event details to avoid conflicts.	**Students & Lecturers**: Receive real-time prompts if incorrect information is entered, reducing errors. **Administrators**: Can trust that bookings are accurate and consistent with policies.

Non-Functional Requirements

Requirement	Description	Impact on User Bases
Performance	The system should deliver quick responses for all user types. It must process bookings and generate reports efficiently, especially during high-demand times.	**Students & Lecturers**: Minimal delays when searching for venues and submitting bookings. **Administrators**: Fast data processing for report generation and management.
Scalability	The system should handle growing numbers of users and bookings as the university expands.	**Students & Lecturers**: Users can still access the system during peak times without issues. **Administrators**: Can manage large volumes of data and bookings efficiently.
Usability	The system must have an intuitive interface for all user types, ensuring ease of use.	**Students & Lecturers**: User-friendly design helps non-tech-savvy users navigate easily. **Administrators**: Efficient workflows for managing bookings and reports.
Security	The system must protect user data and ensure role-based access.	**Students & Lecturers**: Confidence in data protection and secure access. **Administrators**: Restricted access to sensitive data with strong security controls.
Availability	The system should be accessible 24/7 with minimal downtime.	**Students & Lecturers**: Can access the system anytime, even during late hours. **Administrators**: Continuous availability for monitoring and managing bookings.
Maintainability	The codebase should be easy to maintain for future updates and fixes.	**Students & Lecturers**: Minimal disruptions during maintenance. **Administrators**: Updates should not interfere with essential tasks like report generation.

Addressing the Limitations of the Current Process

Problem	Solution
Double-Booking	Real-time availability checks prevent double bookings.
Efficiency	Automated processes reduce manual booking times.
Capacity Management	The system matches venue size to the required capacity.
Error Reduction	Automated validation prevents common booking errors.
Manual Data Entry	Automated data entry and validation reduce errors and save administrative time.
Complex Approval Processes	Automated approval workflows streamline the booking process, reducing delays and simplifying venue access.

Primary Challenges in Implementation

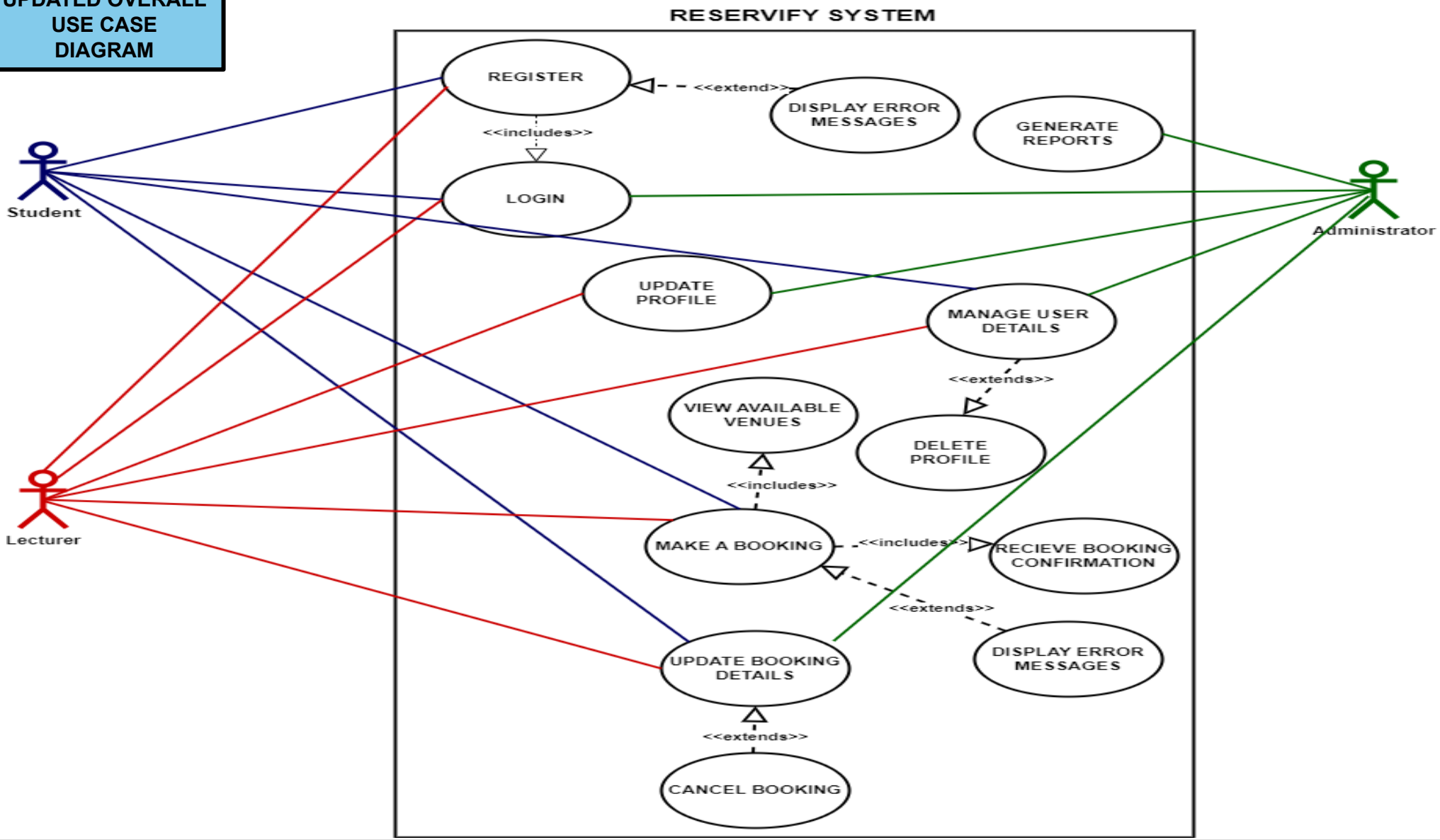
Challenge	Description
Overlapping Bookings	Ensuring the system handles overlapping bookings and prevents double-booking.
User-Friendly Interface	Designing an interface that meets the needs of all users.
Accurate Validations	Implementing feedback mechanisms for user errors.
Integration with Existing Systems	Ensuring compatibility with the university's infrastructure.
User Adoption	Encouraging users to transition from the old manual system to the new platform may face resistance.
Scalability	Ensuring the system can scale effectively as the number of users and venues increases over time.
Change Management	Effectively managing the transition process and addressing any resistance to change from stakeholders.
Maintaining Data Integrity	Automated data validation processes ensure all booking information remains accurate and up to date.

Conclusion

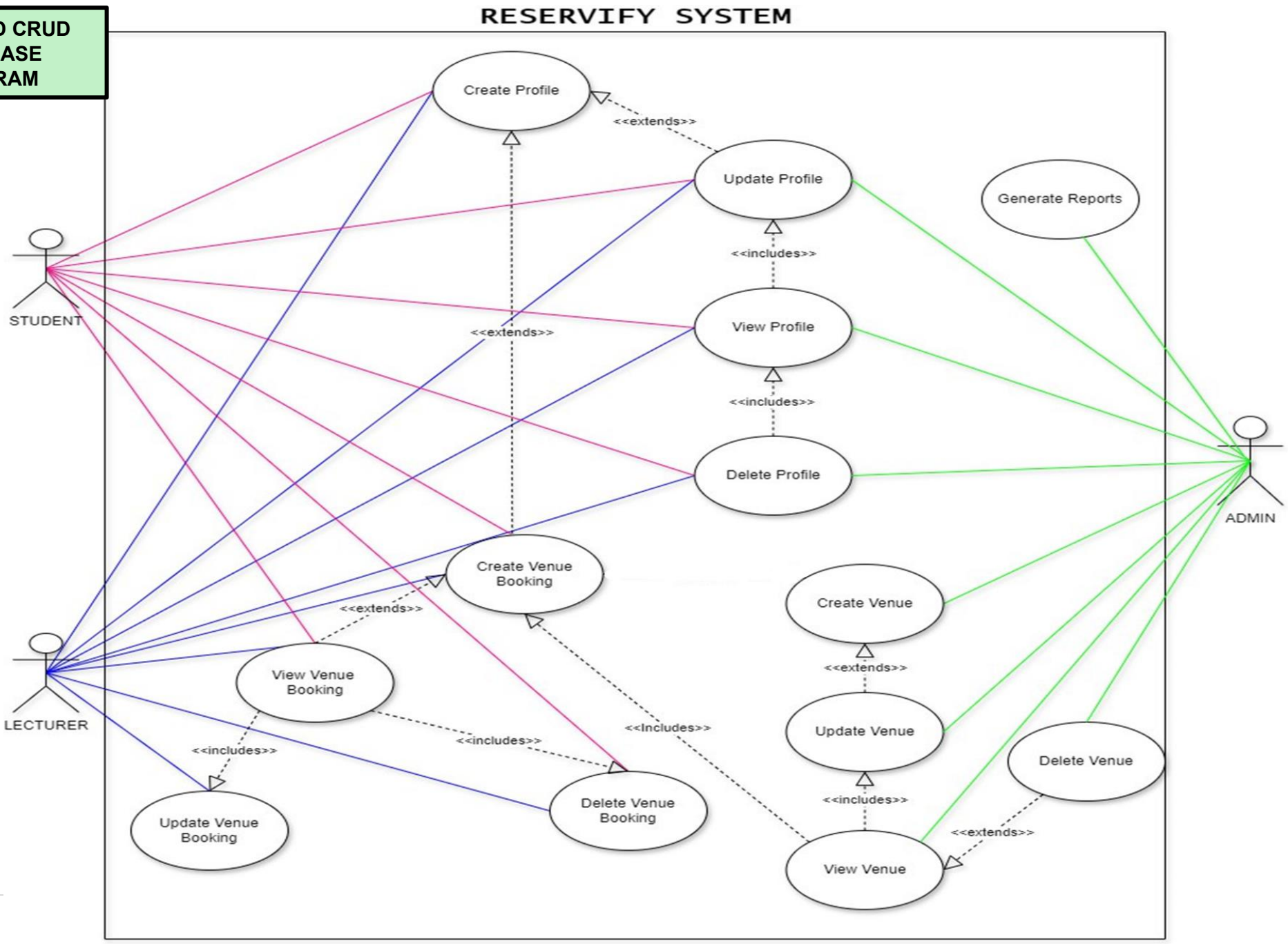
Reservify addresses the critical issues faced in WITS University's current venue booking system by offering an intuitive, automated platform. The system eliminates double bookings, improves efficiency, and offers secure, real-time management of university resources. With scalability, performance, and usability in mind, Reservify transforms the process of managing venues, creating a more organized, efficient academic environment. By catering to students, lecturers, and administrators, it ensures all user needs are met while streamlining campus operations.

2. USE CASE DIAGRAM AND OVERVIEW

UPDATED OVERALL USE CASE DIAGRAM



UPDATED CRUD USE CASE DIAGRAM



USE CASE DESCRIPTION (CREATE BOOKING)

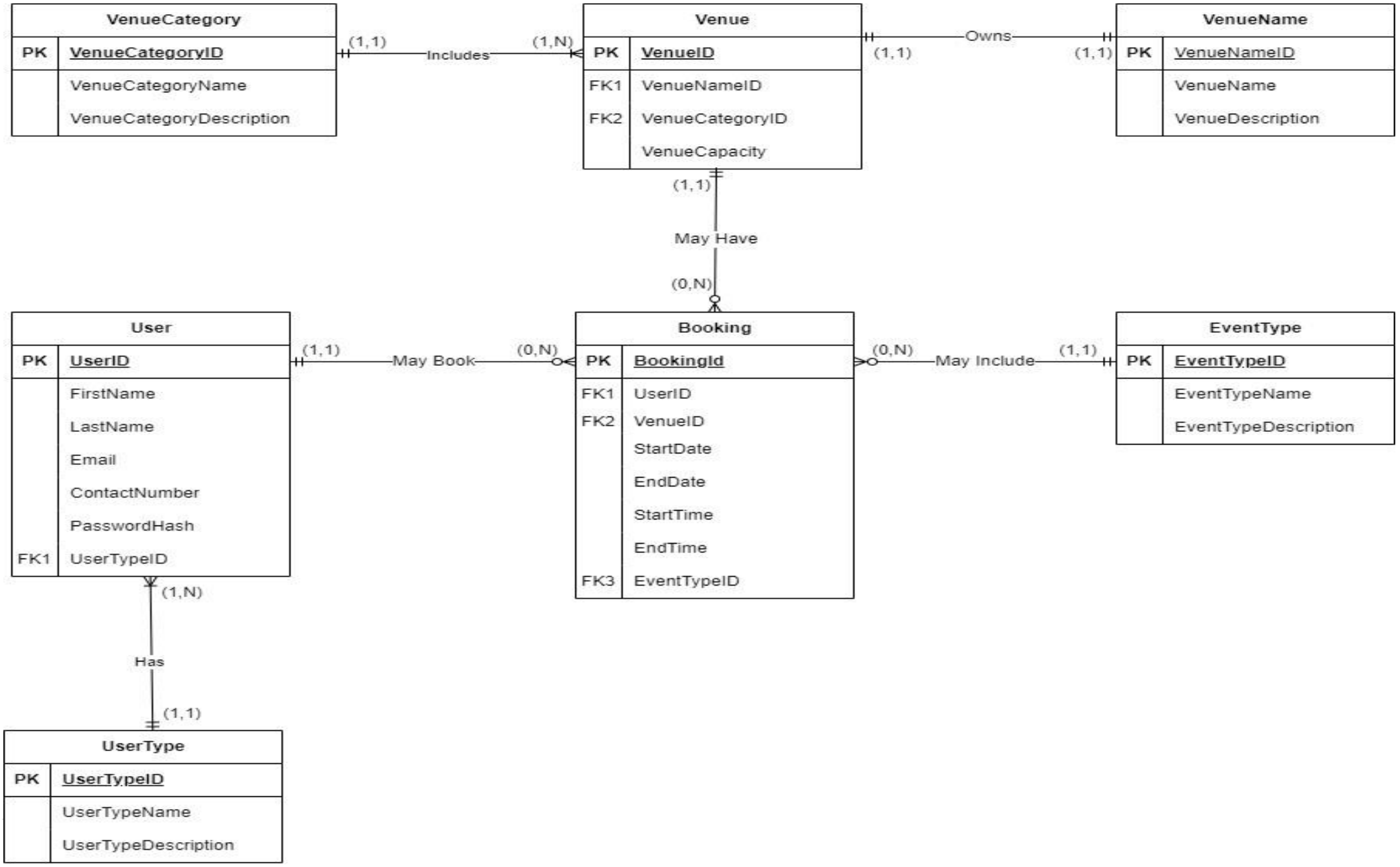
Use Case Name	Create Booking
Use Case Description	The "Create Booking" use case enables lecturers to reserve a venue for their events within the Reservify system. The process begins when a lecturer accesses the system and navigates to the venue booking page. The lecturer can search for available venues based on criteria such as event type, category, capacity, and desired date. Upon initiating the search, the system presents a list of available venues that meet the specified criteria. The lecturer selects a preferred venue from the list and fills out the booking form with relevant details, including event type, venue category, required capacity, date, time, and duration. This ensures that the booking aligns with the lecturer's event needs and preferences. After the lecturer submits the booking request, the system performs a series of validation checks to ensure the accuracy and compliance of the submitted information. Validation includes: • Verifying that all required fields are completed. • Ensuring the event type is appropriate for the selected venue. • Checking that the requested capacity does not exceed the venue's maximum limit. • Confirming that the desired date and time are available. • Validating that the duration of the booking is acceptable If the system detects any errors or conflicts during these checks, it provides specific error messages to guide the lecturer in correcting the issues. This process prevents the submission of invalid bookings, maintaining the integrity of the booking information. Upon successful validation, the system saves the booking details, and the lecturer receives a confirmation message indicating that the venue has been successfully booked. This confirmation includes the booking details, allowing the lecturer to verify that the reservation has been processed correctly. The newly booked information is then reflected in the lecturer's booking history or summary page. In cases of failure, such as system errors or validation issues, the lecturer is informed of the problems and prompted to take corrective actions. This comprehensive approach ensures that the booking process is accurate, user-friendly, and reliable, accommodating the lecturer's needs while maintaining data integrity.
Actor(s)	Lecturer
Trigger	The use case is triggered when the lecturer initiates the venue booking process after logging into the system.
Pre-Conditions	1. The venue booking system, Reservify, must be fully operational and accessible without any system outages. 2. The lecturer has a registered profile on the system, Reservify. 3. The lecturer must be authenticated, having logged into the Reservify system with appropriate rights to create bookings. 4. The lecturer must be on the main menu page within the system.

Main Flow of Activity	Lecturer	System
	1.Navigate to Booking Page The lecturer navigates to the "Create Venue Booking" page.	1.1 The lecturer navigates to the "Create Venue Booking" page. 1.2 The system displays the booking form and relevant guidelines for filling it out. **System Message:** "Welcome to the Venue Booking page. Please fill out the form below to get started."
	2. Search for Available Venues The lecturer enters search criteria (event type, category, capacity, date) to find available venues.	2.1 The system presents a list of venues that match the specified criteria. **System Message:** "Searching for available venues... Please wait."
	3. Select Venue and Input Details The lecturer selects a preferred venue from the list.	3.1 The system displays the selected venue's details and prompts the lecturer to enter additional booking information (event type, venue category, capacity, date, time, duration). **System Message:** "You have selected [Venue Name]. Please enter the booking details below."
	4. Modify Booking Details The lecturer enters the required booking details.	4.1 The system provides real-time validation and feedback, ensuring that inputs are correct and identifying any conflicts immediately. **System Message:** "Validating your entries... Please ensure all fields are filled out correctly."
	5. Submit Booking Request The lecturer clicks the button to submit the booking request, finalizing the details.	5.1 The system validates the submitted information: 5.1.1 Ensures all required fields are completed correctly. 5.1.2 Verifies the new booking details for availability. 5.1.3 Checks for any conflicts with existing bookings. **System Message:** "Submitting your booking request... Please hold on." **6. Handle Validation Outcomes** **6.1 If validation passes:** 6.1.1 The system confirms the booking and stores the details. 6.1.2 A confirmation message is generated for the lecturer. 6.1.3 A confirmation email with the updated booking details is sent to the lecturer. System Message: "Your booking has been successfully created! A confirmation email has been sent to you." **6.2 If validation fails:** 6.2.1 The system displays specific error messages, prompting the lecturer to correct any issues before resubmitting. System Message: "Error: [specific error message]. Please correct the highlighted fields and try again."

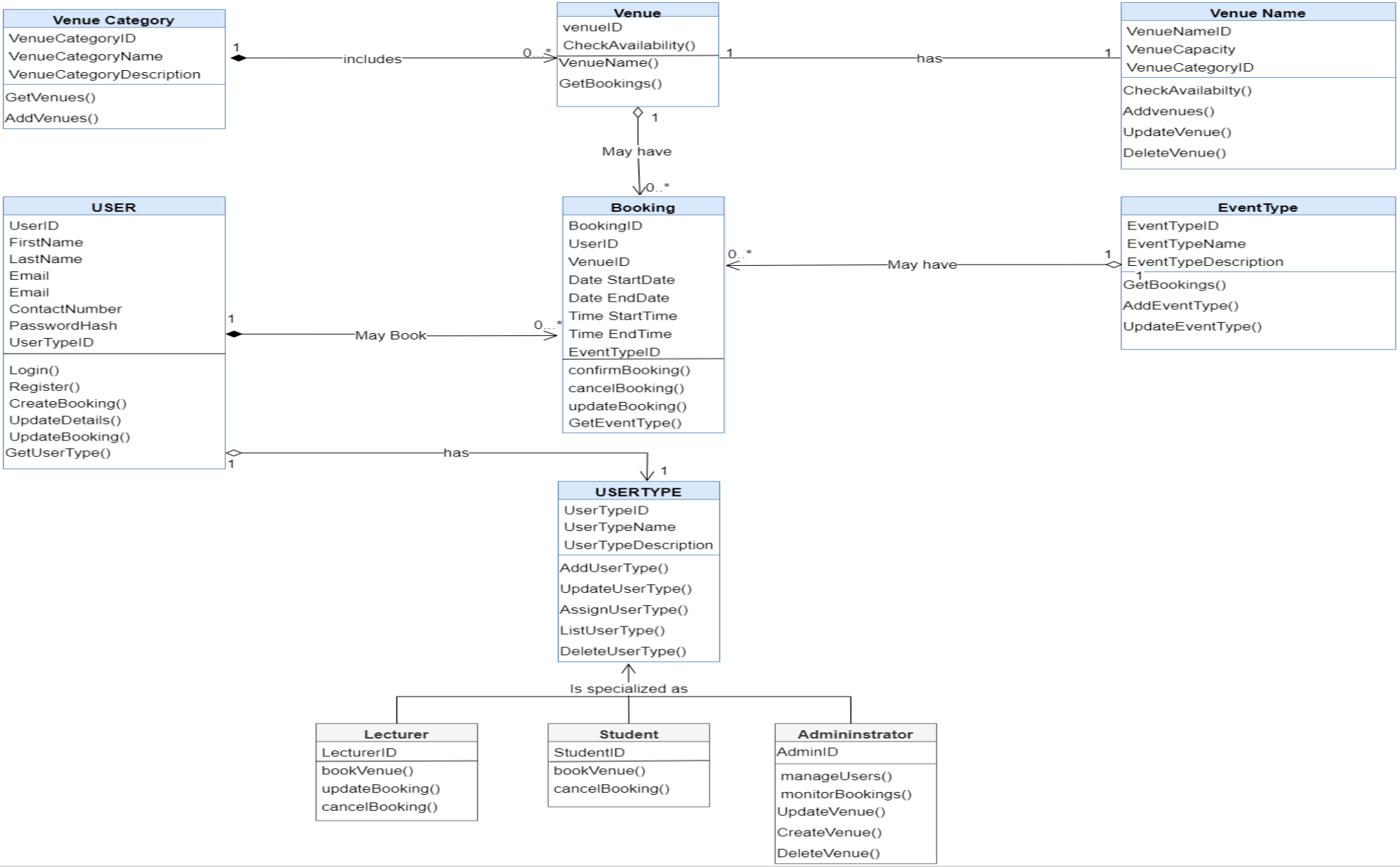
Alternative Flows of Activity	**1. Invalid Booking Data** 1.1 The system detects invalid booking details. 1.2 It displays an error message, prompting the lecturer to correct and resubmit the form. **System Message:** "Please correct the following errors: [list of errors]." 1.3 The lecturer revises and resubmits the form. **2. Venue Not Available** 2.1 The venue selected by the lecturer is not available at the requested date and time. 2.2 The system notifies the lecturer that the venue is not available. **System Message:** "The selected venue is not available for your chosen date and time. Please select another venue or adjust your booking details." 2.3 The lecturer chooses another venue or changes the event date and time.
Post Conditions	**On Success:** • The venue is successfully booked and stored in the system. • The lecturer receives a confirmation of the booking via notification, including the booking details. • The newly booked information is reflected in the lecturer's booking history or summary page. **On Failure:** • The booking process is unsuccessful, and no venue booking is recorded. • The lecturer does not receive a confirmation message and is informed of the errors encountered.

ERD

3. ENTITY RELATIONSHIP DIAGRAM (ERD)

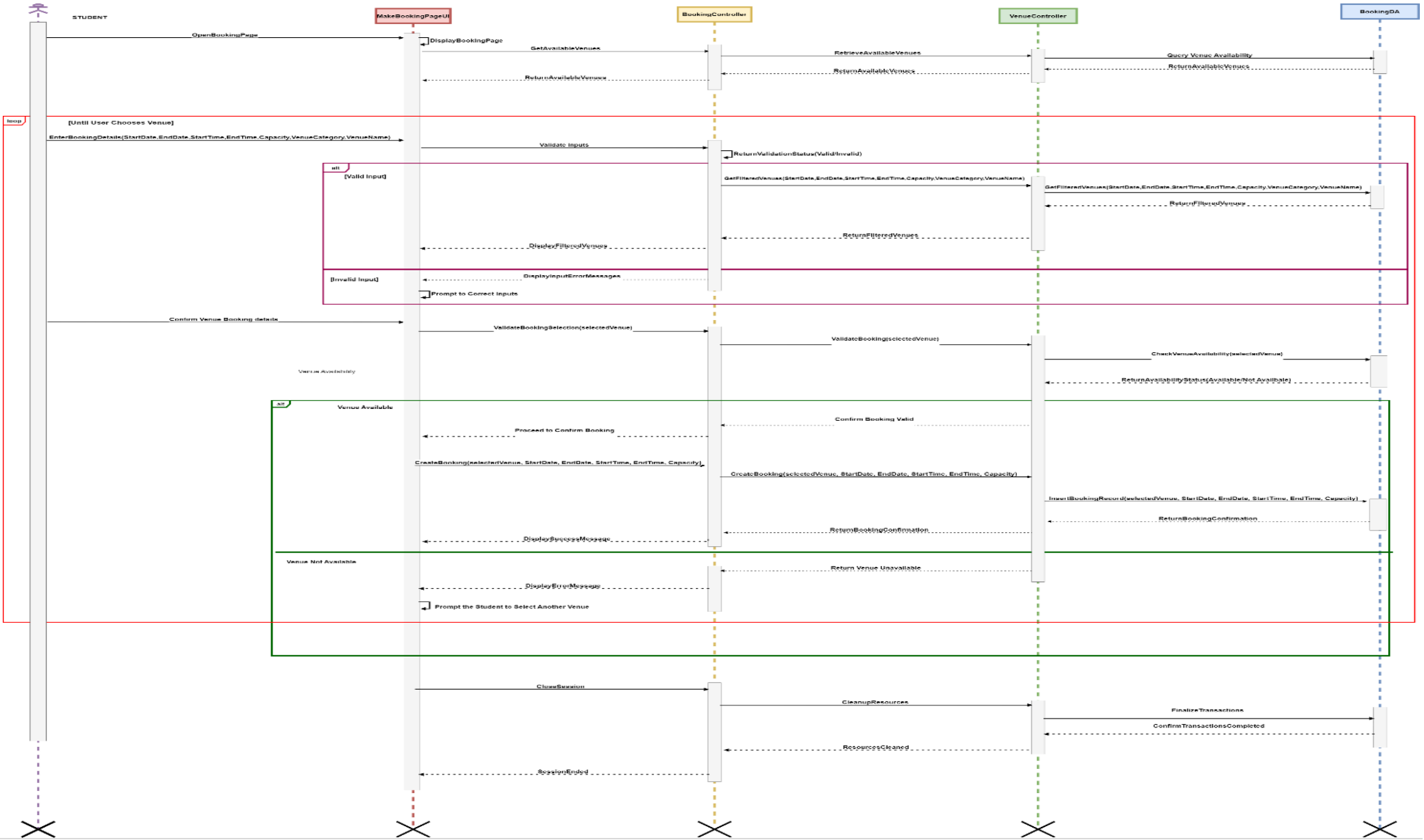


4. DOMAIN MODEL CLASS DIAGRAM

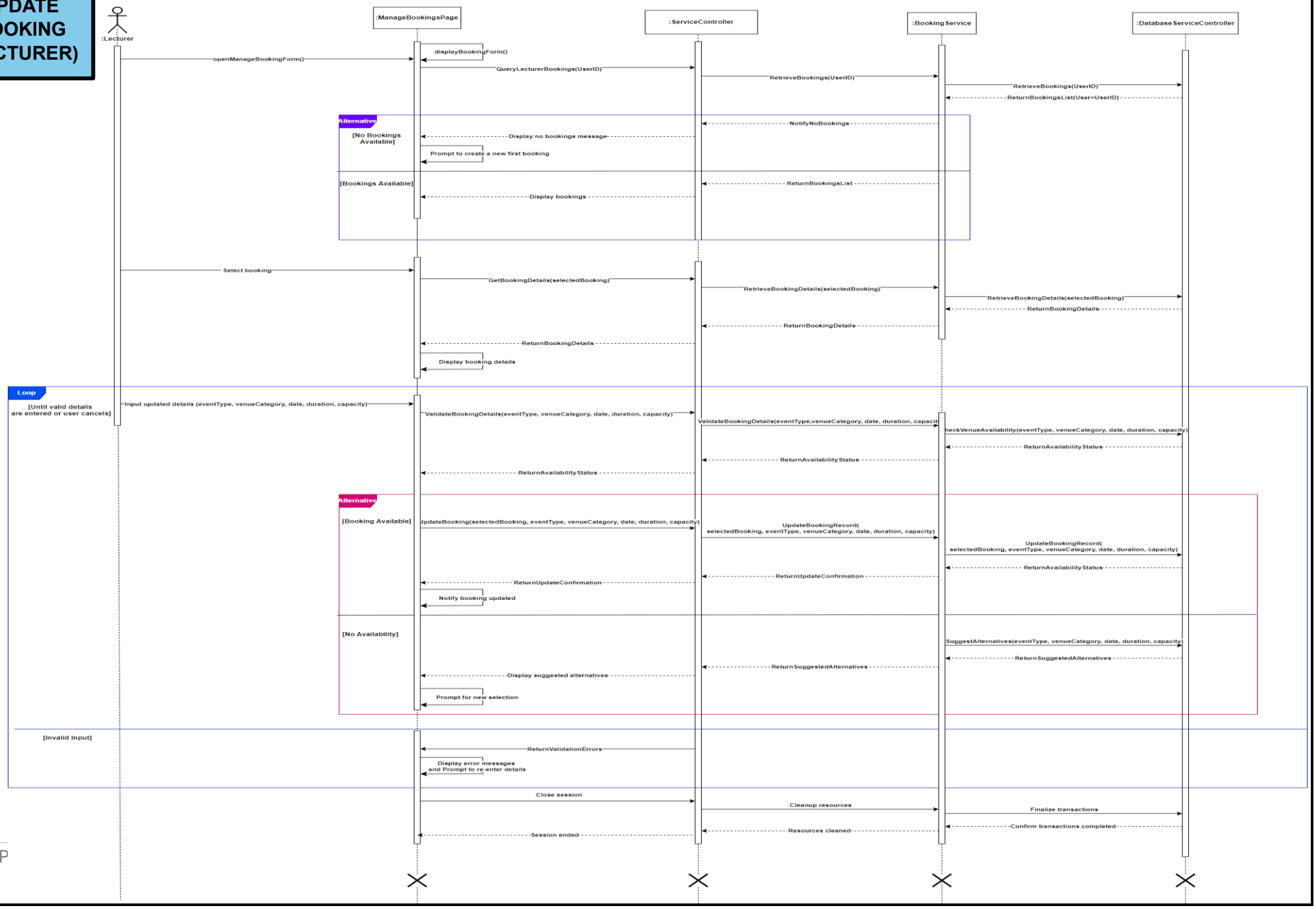


CREATE BOOKING (STUDENT)

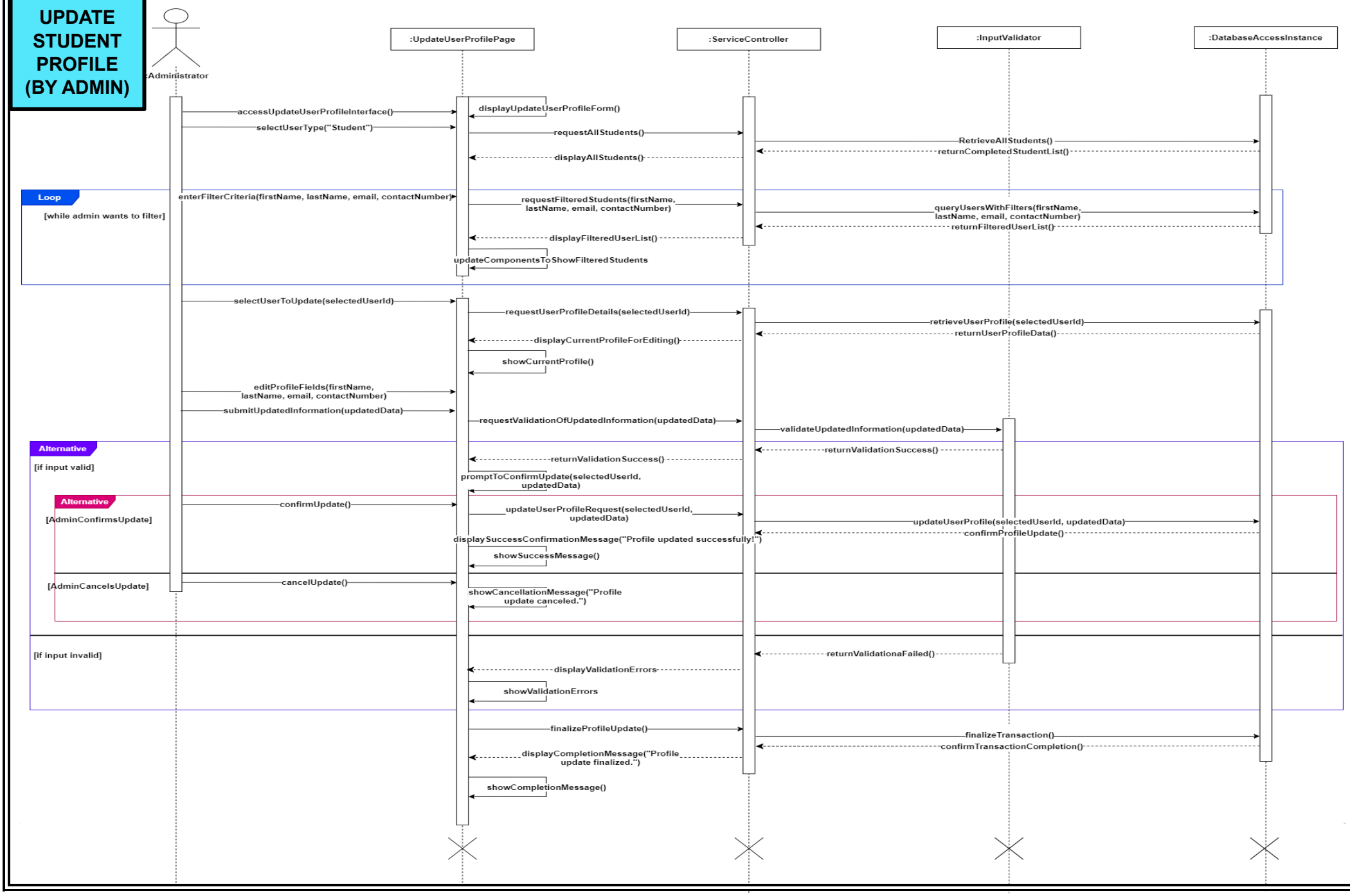
5. USE CASE REALIZATION (SEQUENCE DIAGRAM)



UPDATE BOOKING (LECTURER)



UPDATE STUDENT PROFILE (BY ADMIN)



6. REPORT DESIGN AND STRUCTURE

The **Venue Booking Report** is meticulously crafted to deliver comprehensive insights into venue bookings based on user-defined criteria. This report empowers the **Admin** to filter and analyse booking data effectively, facilitating informed decision-making for venue management. By enabling quick assessments of booking trends and statuses, this report enhances operational efficiency.

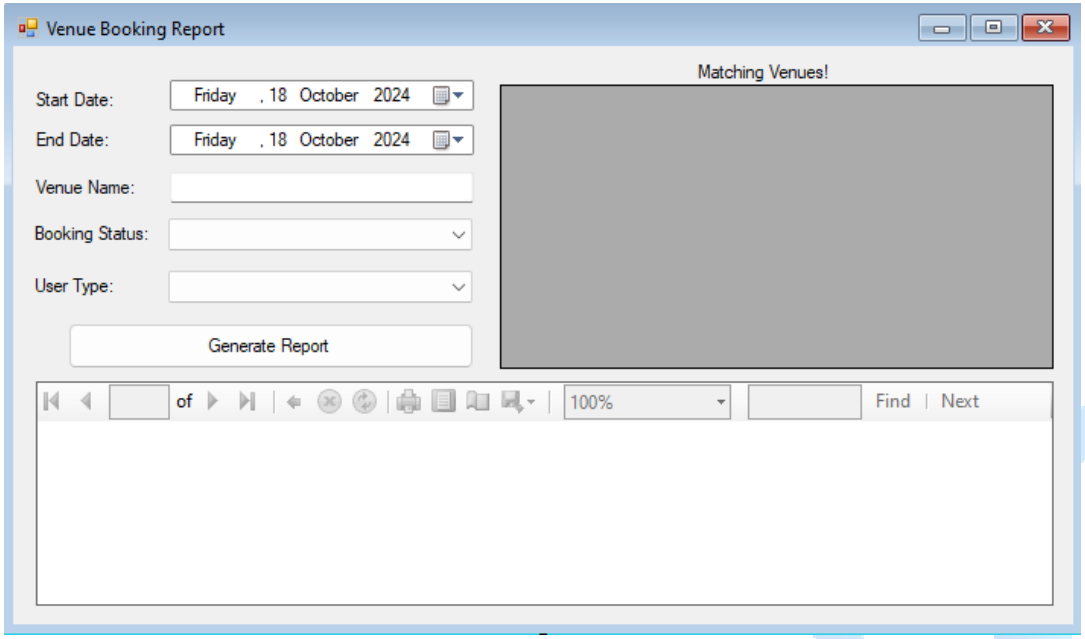
Report Generation Trigger

The report generation process begins when the Admin clicks the "Generate Report" button (**btnGenerateReport**). This **action triggers** the btnGenerateReport_Click event, initiating several important steps:

Validation	The system first checks if a venue has been selected from the available venues displayed in the **dataGridViewVenues**. If no venue is selected, a message prompts the Admin to choose one, ensuring that the report is generated based on valid data.
Parameter Setup	Once a venue is selected, the system gathers the values from the user inputs: 2.1 **Start Date** and **End Date:** Defines the time frame for the report. 2.2 **Selected Venue:** The venue chosen from the grid. 2.3 **Booking Status:** The status of bookings (e.g., Confirmed, Pending, Cancelled). 2.4 **User Type:** The type of user (e.g., Student, Lecturer) for filtering results. These values are then organized as parameters that will be passed to the report.
Report Generation	With the selected venue and specified parameters, the system processes the request and generates the report from the database controller. The report will display booking information based on the criteria set by the Admin, as illustrated in the wireframe.
Presentation	Finally, the generated report is presented to the Admin, allowing them to review and analyse the venue booking data effectively.

Wireframe Components

Filters	1. **Start Date** and **End Date**: Allow the Admin to specify the time frame for bookings. 2. **Venue Name**: Enables searching for specific venues. 3. **Booking Status**: Admin can filter bookings as "Confirmed," "Pending," or "Cancelled." 4. **User Type**: This filter distinguishes between different types of users (e.g., Student, Lecturer).
Report Summary	The report features a summary section that displays total counts for bookings categorized by their status. This summary offers users an at-a-glance view of venue utilization, enabling quick assessments of booking performance and trends.
Booking Details Table	A detailed table lists individual bookings, showcasing critical information including: ✓ Venue Name, User Type, Booking Status, Date of Booking This detailed view allows Admin to track specific transactions, identify patterns in booking behaviour, and make data-driven decisions.
Footer Information	The report concludes with footer information, including metadata about the report generation (e.g., the date created and the user who generated it), providing additional context and accountability for the data presented.



VenueBooking Report

Start Date: [_____] End Date: [_____]

Venue Name: [_____]

Booking Status: [Confirmed/Pending/Cancelled]

User Type: [Student/Lecturer]

Report Summary

Total Bookings: [_____] Cancelled: [_____]

Confirmed: [_____] Pending: [_____]

Booking Details

Venue Name	User Type	Booking Status	Date
Venue A	Student	Confirmed	01/01/2024
Venue B	Lecturer	Pending	02/01/2024
Venue C	Student	Cancelled	03/01/2024

Booking Details

Report Generated On [Date] By [User_Fname]

User Input Collection

Date Pickers	**dtpStartDate** and **dtpEndDate**: These allow users to select the start and end dates for the report, defining the time frame for venue bookings they want to analyse.
Text Input	**txtVenueName**: Users can specify a particular venue they wish to report on.
Combo Boxes	- **cmbBookingStatus**: Users can filter bookings by status (e.g., Confirmed, Pending, Cancelled). - **cmbUserType**: This allows users to filter the report based on different user types (e.g., Student, Lecturer).
Data Grid View	**dataGridViewVenues**: Displays a list of available venues. Users must select a venue to generate the report

Reporting Tool Used: Microsoft Report Definition Language Client Side (RDLC) Reporting Tool.

Reasons for Selection of RDLC Reporting Tool

Integration with Visual Studio	RDLC reports can be easily integrated into .NET applications developed in Visual Studio, allowing for seamless report generation.
Data Binding	RDLC supports robust data binding capabilities, enabling reports to be populated with data directly from various sources, including databases.
Parameterization	The ability to pass parameters to RDLC reports makes it suitable for generating dynamic reports based on user input, enhancing the interactivity of the application.
Flexible Layout	RDLC allows for detailed formatting options, enabling the creation of visually appealing and professional-looking reports.
Ease of Access	RDLC allows for straightforward access to databases, enabling users to execute queries to extract relevant information efficiently.
Enhanced Control	This tool provides greater control compared to many other reporting tools, allowing for more tailored reporting experiences.
Seamless Integration	RDLC is easily integrated with Visual Studio, making it compatible not only with C# but also with other programming languages supported by Visual Studio.
Offline Availability	Reports can be viewed offline, eliminating the need for an internet connection when accessing report data.
Rich Feature Set	The tool includes various features, such as the ability to add timestamps to reports, ensuring accuracy regarding the dates and times reports are generated.
Flexible Data Extraction	Users can execute various SQL statements to specify the exact data to be displayed in the report, enhancing the relevance and utility of the information.
Export Options	RDLC reports can be exported in multiple file formats, including PDF, Word, Excel, and HTML, providing versatility in how reports can be shared and utilized.
Client-Side Management	Being client-side based allows administrators to easily make necessary changes and updates to the information displayed in reports.

Conclusion

The **Venue Booking Report** offers a structured and intuitive approach to data presentation, equipping the Admin with the tools necessary to generate insightful, tailored reports that meet specific needs. By integrating the **Microsoft Report Definition Language Client Side (RDLC)** as the reporting tool, the system ensures a flexible and efficient method for managing and presenting booking information, significantly enhancing the application's overall functionality. This robust reporting capability enables the Admin to effectively monitor and analyse venue bookings within the **Reservify** system, leading to informed decision-making and improved venue management. Ultimately, the Venue Booking Report not only streamlines the reporting process but also contributes to the operational efficiency of venue management.

7. TEST CASE SCENARIOS

CREATE BOOKING (STUDENT)

TEST CASE NUMBER	1	
TEST SCENARIO	**Verify Successful Loading of Available Venues** To verify the successful loading of available venues upon opening the "Create Booking" page	
TEST STEPS	1. The student navigates to the "Create Booking" page. 2. The system retrieves available venues from the database. 3. The list of available venues is displayed to the user.	
PREREQUISITES	1. The booking system (Reservify) is operational and accessible. 2. The student is logged in and has the necessary permissions to make a booking. 3. There must be available venues in the database.	
TEST DATA	Student, UserID, Password, Available Venues	
VALUES FOR TEST DATA	1. Student Name: James Dean 2. UserID: 1001 3. Available Venues: Room 101, Room 102, Room 103	
EXPECTED RESULTS	A list of available venues is displayed to the user	
ACTUAL RESULTS	The system successfully queried the database and returned a list of available venues, displaying each venue's details such as name, capacity, and categories.	
TEST STATUS	**PASS**	
	The system successfully completes the booking process, ensuring all required fields are validated and the booking is created without errors.	
	FAIL	
	Type of Failure	**Solution**
	1. No available venues displayed after the page loads	Verify that there are available venues in the database. Check database connection and query logic.
	2. System crash during retrieval	Investigate performance issues and review error logs for query processing exceptions.
	3. Incorrect venue details displayed	Confirm that the system fetches the correct venue data from the database, including all necessary details.
	4. system crash or long delays when loading the available venues	Review system performance, focusing on query execution time and server load during data retrieval.

TEST CASE NUMBER	2
TEST SCENARIO	**Verify Successful Dynamic Filtering of Venues with Valid Inputs** To verify successful dynamic filtering of venues with valid inputs
TEST STEPS	1. Student enters valid details for Start Date, End Date, Start Time, End Time, Capacity, Venue Category 2. The system validates the entered details. 3. If inputs are valid, the system filters venues based on the criteria. 4. The filtered venues matching the inputs are displayed to the user.
PREREQUISITES	1. The booking system (Reservify) is operational and accessible 2. Valid venue details must be entered by the user.
TEST DATA	Start Date, End Date, Start Time, End Time, Capacity, Venue Category, Venue Name
VALUES FOR TEST DATA	1. Start Date: 2024-12-01 2. End Date: 2024-12-02 3. Start Time: 08:00 4. End Time: 17:00 5. Capacity: 50 6. Venue Category: Lecture Hall 7. Venue Name: FNB 141
EXPECTED RESULTS	A list of new updated available venues is displayed to the user based on their search.
ACTUAL RESULTS	The system should successfully query the database and return a list of available venues, displaying each venue's details such as name, capacity, and category.
TEST STATUS	**PASS** The system accurately processes the user input and confirms the booking, displaying a success message and updating the venue's availability accordingly.

FAIL

Type of Failure	Solution
1. No filtered venues displayed	Ensure that venues matching the criteria exist in the database and that the filtering logic is working correctly.
2. Incorrect venues displayed	Verify that the input validation and filtering criteria are accurate. Ensure the query filters venues based on the right fields.
3. System error or crash	Review performance logs and check for errors in the filtering logic.
4. No venues are returned despite valid criteria	Confirm that the venue data exists in the database and ensure that the dynamic filtering logic is correctly implemented to retrieve venues based on the provided criteria.

TEST CASE NUMBER	3	
TEST SCENARIO	**Verify Error Handling for Invalid Venue Details** To verify error handling when invalid venue details are entered.	
TEST STEPS	1. User enters invalid details, such as End Date earlier than Start Date or negative capacity. 2. The system validates the inputs. 3. If inputs are invalid, an error message is displayed. 4. The user is prompted to correct the invalid inputs.	
PREREQUISITES	1. The booking system (Reservify) is operational and accessible 2. Invalid venue details provided by the user.	
TEST DATA	Start Date, End Date, Start Time, End Time, Capacity, Venue Category, Venue Name	
VALUES FOR TEST DATA	1. Start Date: 2023-06-03 2. End Date: 2023-06-02 3. Start Time: 08:00 4. End Time: 17:00 5. Capacity: 50000 6. Venue Category: - 7. Venue Name: FNB 500	
EXPECTED RESULTS	The system displays an error message and prompts the user to correct the inputs.	
ACTUAL RESULTS	The system correctly displayed error messages for invalid date and capacity inputs.	
TEST STATUS	**PASS**	
	The system effectively checks venue availability and prevents booking conflicts by displaying appropriate messages and guiding the user to choose available options.	
	FAIL	
	Type of Failure	**Solution**
	1. No error message displayed	Verify that the validation logic for dates and capacity is correctly implemented.
	2. System accepts invalid input	Check the validation rules to ensure incorrect inputs are not allowed to proceed further.
	3. System crashes during validation	Investigate the error handling during input validation to ensure the system can gracefully manage unexpected inputs without crashing or hanging.

CREATE BOOKING (STUDENT)

TEST CASE NUMBER	4
TEST SCENARIO	**Successful Venue Selection and Booking Confirmation** To verify successful venue selection and booking confirmation when the venue is available
TEST STEPS	1. Student selects a venue and confirms the booking. 2. The system checks venue availability in the database. 3. If the venue is available, the system confirms the booking and records it. 4. A success message is displayed to the user.
PREREQUISITES	1. The booking system (Reservify) is operational and accessible 2. The selected venue must be available at the time of booking.
TEST DATA	Selected Venue, Start Date, End Date, Start Time, End Time, Capacity, Venue Category, Venue Name
VALUES FOR TEST DATA	1. Selected Venue: Room 101 2. Start Date: 2024-12-01 3. End Date: 2024-12-01 4. Start Time: 09:00 5. End Time: 17:00 6. Capacity: 150 7. Venue Category: Lecture Hall 8. Venue Name: FNB 141
EXPECTED RESULTS	The system confirms the booking, records it in the database, and displays a success message to the user.
ACTUAL RESULTS	The system successfully confirmed the booking and displayed a success message
TEST STATUS	**PASS**
	The system operates seamlessly, performing the required validation and booking operations, while providing clear feedback to the user at each step.
	FAIL

Type of Failure	**Solution**
1. Booking not confirmed	Verify that the venue is available, and that the booking logic correctly processes the request
2. System failure during booking confirmation	Investigate the booking process and check logs for errors that might have occurred during confirmation.

CREATE BOOKING (STUDENT)

TEST CASE NUMBER	5
TEST SCENARIO	**Verify Error Handling for Unavailable Venues** To verify error handling when the selected venue is unavailable
TEST STEPS	1. User selects a venue and confirms the booking 2. The system checks venue availability in the database 3. If the venue is unavailable, the system displays an error message. 4. The user is prompted to select another venue.
PREREQUISITES	1. The booking system (Reservify) is operational and accessible 2. The selected venue must be unavailable at the time of booking.
TEST DATA	Selected Venue, Start Date, End Date, Start Time, End Time, Capacity
VALUES FOR TEST DATA	2. Selected Venue: Room 102 3. Start Date: 2024-12-01 4. End Date: 2024-12-01 5. Start Time: 09:00 6. End Time: 17:00 7. Capacity: 50
EXPECTED RESULTS	The system displays an error message and prompts the user to select another venue.
ACTUAL RESULTS	The system correctly displayed an error message indicating that the selected venue was unavailable.
TEST STATUS	**PASS**
	The system handles the booking scenario efficiently, verifying venue availability and updating the database without any issues, ensuring no double-booking occurs
	FAIL

Type of Failure	**Solution**
1. No error message displayed:	Ensure the system is properly checking venue availability and displays relevant feedback.
2. System allows booking despite venue unavailability:	Review the venue availability check and ensure the system prevents double bookings.

CREATE BOOKING (STUDENT)

TEST CASE NUMBER	6
TEST SCENARIO	**Verify that a venue booking fails if the venue is unavailable** To verify that a venue booking fails if the venue is unavailable during the requested time slot
TEST STEPS	1. User navigates to the "Create Booking" page. 2. The user enters booking details, including Start Date, End Date, Start Time, End Time, Capacity, and Venue Category. 3. The student selects a venue and confirms the booking. 4. The system checks the venue's availability in the database. 5. If the venue is unavailable, the system displays an error message and prompts the user to choose another venue.
PREREQUISITES	1. The booking system (Reservify) is operational and accessible. 2. The student is logged in and has the necessary permissions to book a venue. 3. The selected venue must already be booked during the requested time.
TEST DATA	Start Date, End Date, Start Time, End Time, Capacity, Venue Category, Venue Name.
VALUES FOR TEST DATA	1. Start Date: 2024-12-01 2. End Date: 2024-12-01 3. Start Time: 09:00 4. End Time: 17:00 5. Capacity: 50 6. Venue Category: Lecture Hall 7. Venue Name: Room 101
EXPECTED RESULTS	The system displays an error message stating that the venue is unavailable and prompts the user to select another venue.
ACTUAL RESULTS	The system correctly displays the venue's unavailability and prompts the user to choose a different venue.
TEST STATUS	**PASS**
	The system executes the booking logic correctly, ensuring that the student is informed of venue unavailability and prompted to select another time slot or venue.
	FAIL

Type of Failure	**Solution**
1. Venue availability check fails	Review the venue availability logic and ensure the system is checking the correct time slots.
2. System crash or hang during booking attempt	Investigate potential performance issues or bugs related to the venue availability check and booking process.

TEST CASE NUMBER	7	
TEST SCENARIO	**Proper Session Cleanup after Failed Booking** Ensure proper session cleanup after a failed booking attempt	
TEST STEPS	1. After a failed booking (e.g., venue unavailable), the student logs out or closes the "Create Booking" page. 2. The system ensures that no pending transactions or resources are left unhandled. If the venue is unavailable, the system displays an error message. 3. The session is successfully closed.	
PREREQUISITES	1. The booking system (Reservify) is operational and accessible 2. The selected venue must be unavailable at the time of booking...	
TEST DATA	Selected Venue, Start Date, End Date, Start Time, End Time, Capacity	
VALUES FOR TEST DATA	1. Selected Venue: Room 102 2. Start Date: 2024-12-01 3. End Date: 2024-12-01 4. Start Time: 09:00 5. End Time: 17:00 6. Capacity: 50 7. Venue Category: Lecture Hall 8. Venue Name: FNB 141	
EXPECTED RESULTS	The system should finalize and clean up all resources and transactions properly, regardless of booking success.	
ACTUAL RESULTS	The system cleaned up all resources and closed the session without errors.	
TEST STATUS	**PASS**	
	The system successfully clears all session data and terminates any pending transactions after the failed booking attempt, ensuring no lingering data or resource locks remain. The user is logged out or redirected as expected, with the system maintaining proper session integrity	
	FAIL	
	Type of Failure	**Solution**
	1. Session remains active or resources left unhandled after closure	Verify that the session management properly terminates all transactions and ensure that resources are closed and cleaned up when the user logs out or closes the page.
	2. Error during session cleanup	Review error logs and resource handling processes to identify where the cleanup failed or encountered issues.
	3. Failed booking session data persists across different sessions	Validate that session data associated with the failed booking is properly discarded and not carried over into subsequent user sessions.

TEST CASE NUMBER	1	
TEST SCENARIO	**Retrieve Bookings** Ensure that the system accurately retrieves and displays a list of bookings for a logged-in lecturer.	
TEST STEPS	5. Lecturer navigates to the Manage Bookings page. 6. System automatically queries for the lecturer's bookings. 7. System displays a loading indicator while retrieving data. 8. System displays the retrieved list of bookings.	
PREREQUISITES	1. The venue booking system, Reservify, is operational and accessible. 2. User is logged in as a lecturer with appropriate permissions. 3. There exists Existing bookings for the lecturer. 4. There exists an identifiable booking that the lecturer intends to update.	
TEST DATA	UserID	
VALUES FOR TEST DATA	User ID: 101 (internal numeric ID assigned at login/registration)	
EXPECTED RESULTS	1. The system successfully queries the database for the lecturer's bookings. 2. The system displays a list of bookings associated with the lecturer's UserID, clearly indicating booking details such as event type, date, and venue.	
ACTUAL RESULTS	(To be filled after execution) The system displayed 3 bookings for the lecturer with a UserID of 101, including relevant details for each booking.	
TEST STATUS	**PASS**	
	The lecturer can view their bookings, confirming that the retrieval function works correctly and meets user expectations.	
	FAIL	
	Type of Failure	**Solution**
	1. No bookings displayed.	Verify database connections, ensure that bookings exist for the lecturer, and review the query logic.
	2. System crashes or hangs during retrieval.	Investigate performance issues, check for exceptions in the query processing logic, and monitor system logs.
	3. Incorrect bookings displayed (e.g., others' bookings).	Verify the filtering logic used to retrieve bookings, ensuring data integrity and accuracy.
	4. Error message displayed instead of bookings.	Review error handling mechanisms in the system for exceptions raised during the data retrieval process.

UPDATE BOOKING (LECTURER)

TEST CASE NUMBER	2
TEST SCENARIO	**No Bookings Available** Ensure that the system properly handles the situation where a logged-in lecturer has no existing bookings.
TEST STEPS	1. Lecturer navigates to the "Manage Bookings" page. 2. The system automatically queries the database for the lecturer's bookings upon page load. 3. The system displays a loading indicator while retrieving data. 4. The system checks the database for any existing bookings associated with the lecturer's UserID.
PREREQUISITES	1. The venue booking system, Reservify, is operational and accessible without outages. 2. The lecturer is logged in as a lecturer with appropriate permissions. 3. There are no existing bookings in the system for the lecturer.
TEST DATA	**UserID**
VALUES FOR TEST DATA	**UserID**: 102 (internal numeric ID assigned at login/registration)
EXPECTED RESULTS	1. The system successfully queries the database for the lecturer's bookings. 2. The system displays a message indicating that no bookings are available, along with a prompt encouraging the lecturer to create a new booking.
ACTUAL RESULTS	(To be filled after execution) The system displayed: "No bookings found. Would you like to create a new booking?"
TEST STATUS	**PASS**
	The lecturer is informed appropriately and can proceed to create a new booking.
	FAIL

Type of Failure	**Solution**
1. Message not displayed when no bookings exist.	Debug the logic that checks for existing bookings to ensure it accurately reflects the current state.
2. System hangs or crashes during the query.	Investigate performance issues in the query processing logic and ensure proper error handling is in place.
3. Incorrect message displayed (e.g., shows other users' bookings).	Verify the logic that fetches and displays bookings is functioning correctly, ensuring user-specific filtering.
4. No prompt to create a new booking appears.	Ensure the system triggers the prompt correctly when no bookings are found, verifying the condition checks.

<table>
<tr><td>TEST CASE NUMBER</td><td colspan="2">3</td></tr>
<tr><td>TEST SCENARIO</td><td colspan="2"><u>Successful Booking Update</u>
Ensure that a lecturer can successfully update an existing booking with valid details.</td></tr>
<tr><td>TEST STEPS</td><td colspan="2">5. Lecturer selects an existing booking to update from the list of their bookings.
6. The system displays the current booking details in an editable format.
7. Lecturer inputs new event details.
8. Lecturer clicks the "Update" button to submit the changes.</td></tr>
<tr><td>PREREQUISITES</td><td colspan="2">1. The venue booking system, Reservify, is operational and accessible.
2. The lecturer is logged in with User Type lecturer.
3. There is at least one existing booking in the available list for the lecturer to update.</td></tr>
<tr><td>TEST DATA</td><td colspan="2">Variables: Venue Name, Venue Category, Date, Duration, Capacity</td></tr>
<tr><td>VALUES FOR TEST DATA</td><td colspan="2">Selected Booking ID: 201
New Details:
• Venue Name: Room 101
• Venue Category: Conference Room
• Date: 2024-11-01
• Duration: 2 hours
• Capacity: 50</td></tr>
<tr><td>EXPECTED RESULTS</td><td colspan="2">1. The system updates the booking with the new details.
2. The System displays a success message confirming the update.</td></tr>
<tr><td>ACTUAL RESULTS</td><td colspan="2">1. The lecturer received a confirmation message: "Your booking has been successfully updated."
2. The updated booking details are correctly displayed on the lecturer's booking page showing “New Details”</td></tr>
<tr><td rowspan="6">TEST STATUS</td><td colspan="2">PASS</td></tr>
<tr><td colspan="2">The update functionality works correctly, and the lecturer receives confirmation of the update.</td></tr>
<tr><td colspan="2">FAIL</td></tr>
<tr><td>Type of Failure</td><td>Solution</td></tr>
<tr><td>1. Update fails without an error message.</td><td>Verify the update logic and ensure the system handles errors properly, providing feedback to the user.</td></tr>
<tr><td>2. Database write permissions are denied.</td><td>Check the database configurations and permissions to ensure the lecturer has the rights to update bookings.</td></tr>
<tr><td>3. Incorrect details displayed after update.</td><td>Validate the logic that retrieves booking details to ensure it reflects the most recent information.</td></tr>
<tr><td>4. System crashes during the update process.</td><td>Investigate performance issues and check for unhandled exceptions in the update logic.</td></tr>
</table>

UPDATE BOOKING (LECTURER)

TEST CASE NUMBER	4
TEST SCENARIO	**Booking Update with No Availability** Verify the system's ability to handle booking updates when no venue is available for the requested details, ensuring proper user feedback and alternative suggestions.
TEST STEPS	1. Lecturer selects a booking to update. 2. Inputs new event details that conflict with existing bookings. 3. Clicks "Update."
PREREQUISITES	1. The venue booking system, Reservify, is operational and accessible without any downtime. 2. The lecturer is logged in with appropriate permissions as a lecturer. 3. At least one existing booking is present in the lecturer's account. 4. There are no available venues that match the new booking details for the specified date.
TEST DATA	• **Venue Name** • **Venue Category** • **Date** • **Duration** • **Capacity**
VALUES FOR TEST DATA	**1.Selected Booking ID**: 202 (internal identifier for the booking) 2. **New Details**: • **Venue Name**: "Room 101" • **Venue Category**: "Lecture Room" • **Date**: "2024-11-01" • **Duration**: "2 hours" • **Capacity**: "50"
EXPECTED RESULTS	1. The system processes the update request and initiates a check for venue availability. 2. If no venues are available for the requested date and details 2.1 The system displays a clear message: "No availability for the selected venue on the requested date. Here are some alternative venues available for your booking." 2.2. The system should provide a list of alternative venues that are available for the requested date and time, including their details (e.g., name, capacity, category).
ACTUAL RESULTS	Example result: "No availability for the selected venue. Suggested alternatives include: Room 102, Capacity: 60; Hall A, Capacity: 80."

TEST STATUS	PASS	
	The system correctly identifies the lack of venue availability and provides alternatives, ensuring a user-friendly experience.	
	FAIL	
	Type of Failure	**Solution**
	1. No Availability Message Not Displayed	1. Review venue availability checking logic and ensure alternative suggestions are generated. 2.Ensure that error handling mechanisms are in place to manage this scenario gracefully.
	2. Alternative Suggestions Not Provided	1.Verify the logic that generates alternative venue suggestions. Ensure it is triggered correctly when the initial venue is unavailable. 2. Validate that the alternative suggestions are accurate and relevant to the requested date.
	3.System Crashes or Freezes During Update Process:	1. Investigate system performance, particularly during high-load scenarios, and check for any unhandled exceptions in the booking update logic. 2. Monitor system logs for errors during the update request to identify potential bottlenecks or issues.
	4.Inconsistent Data Displayed:	1. Ensure that data retrieval logic accurately reflects the most recent booking information in both the update form and the suggestions provided. 2. Cross-verify that database queries return the expected data in real-time.

UPDATE BOOKING (LECTURER)

TEST CASE NUMBER	5
TEST SCENARIO	**Invalid Input for Booking Update** Ensure that the system validates input correctly when a lecturer attempts to update a booking with invalid details, providing appropriate feedback.
TEST STEPS	1. Lecturer logs into the Reservify system. 2. Navigates to the "Manage Bookings" page. 3. Selects a booking they wish to update from the list of existing bookings. 4. Input Invalid Event Details. 5. The lecturer confirms the changes by clicking the "Update" button. 6. The system processes the update request and performs validations on the input fields
PREREQUISITES	1. The venue booking system, Reservify, is operational and accessible without any downtime. 2. The lecturer is logged in with appropriate permissions as a lecturer. 3. At least one existing booking is present in the lecturer's account.

<table>
<tr><td>TEST DATA</td><td>

- **Venue Name**
- **Venue Category**
- **Date**
- **Duration**
- **Capacity**

</td></tr>
<tr><td>VALUES FOR TEST DATA</td><td>1. Selected Booking ID: 203 (internal identifier for the booking)
2. New Details:
• Venue Name: "Room 101"
• Venue Category: "Lecture Room"
• Date: "2022-01-01" (invalid as it's a past date)
• Duration: "2 hours"
• Capacity: "50"</td></tr>
<tr><td>EXPECTED RESULTS</td><td>1. The system should not proceed with the update due to invalid input.
2. The system displays validation error messages clearly indicating the issues:
2.1 "The selected date cannot be in the past. Please choose a valid future date."
2.2 Any additional validation errors related to other fields, if applicable.</td></tr>
<tr><td>ACTUAL RESULTS</td><td>(To be filled after execution)
Example result: "The selected date cannot be in the past. Please choose a valid future date."</td></tr>
<tr><td>TEST STATUS</td><td>
<table>
<tr><th colspan="2">PASS</th></tr>
<tr><td colspan="2">The system correctly identifies the invalid input and informs the lecturer, ensuring input validation works as intended.</td></tr>
<tr><th colspan="2">FAIL</th></tr>
<tr><th>Type of Failure</th><th>Solution</th></tr>
<tr><td>1. Validation Errors Not Displayed</td><td>1. Review the validation logic in the update function to ensure that it checks for past dates and returns appropriate error messages.
2. Test other edge cases for input validation to confirm comprehensive coverage.</td></tr>
<tr><td>2. System Crashes or Freezes During Validation:</td><td>1. Investigate the performance of the validation logic, particularly for edge cases, and check for unhandled exceptions.
2. Monitor system logs during the validation process to identify any unexpected behavior.</td></tr>
<tr><td>3. Inconsistent Error Messaging:</td><td>1. Ensure that all validation messages are clear, concise, and provide actionable feedback for the user.
2. Validate that error messages are displayed in the user interface in a user-friendly manner.</td></tr>
<tr><td>4. Validation Logic Bypassed:</td><td>1. Review the code to ensure that all fields undergo validation before submission is processed.
2. Conduct unit tests on the validation functions to ensure they are functioning as expected across various scenarios.</td></tr>
</table>
</td></tr>
</table>

UPDATE BOOKING (LECTURER)

TEST CASE NUMBER	6
TEST SCENARIO	**Cancel Booking** Ensure that a lecturer can successfully cancel an existing booking and that the system reflects this change accurately.
TEST STEPS	1. **The lecturer logs into the Reservify system and s**elects the booking they wish to cancel from the list of existing bookings. 2. Initiate Cancellation: Clicks on the "Cancel Booking" button associated with the selected booking. 3. Confirm Cancellation, A confirmation dialog appears asking the lecturer to confirm the cancellation. The lecturer clicks "Yes" or "Confirm" to proceed with the cancellation.
PREREQUISITES	1. The venue booking system, Reservify, is operational and accessible without any downtime. 2. The lecturer is logged in with appropriate permissions as a lecturer. 3. At least one existing booking is present in the lecturer's account.
TEST DATA	**Selected Booking ID**
VALUES FOR TEST DATA	**Selected Booking ID**: 204 (internal identifier for the booking)
EXPECTED RESULTS	1. The system successfully cancels the selected booking. 2. A confirmation message is displayed to the lecturer, stating: "Your booking has been successfully cancelled." 3. The cancelled booking no longer appears in the lecturer's list of active bookings.
ACTUAL RESULTS	(To be filled after execution) Example result: "Your booking has been successfully cancelled."
TEST STATUS	**PASS** The cancellation functionality works as intended, and the booking is removed from the lecturer's account.

FAIL

Type of Failure	Solution
1. Cancellation Does Not Reflect in the System:	1. Verify the cancellation logic in the system to ensure that the database updates correctly after cancellation. 2. Confirm that the booking record is removed or marked as cancelled in the database.
2. Error During Cancellation Process:	1. Investigate any errors that arise during the cancellation request and ensure proper error handling is implemented. 2. Review logs for exceptions or issues that might occur during the cancellation process.
3. Confirmation Message Not Displayed:	1. Ensure that the user interface displays the appropriate confirmation message after a successful cancellation.

		2. Test that confirmation messages are consistent and clear across various cancellation scenarios.
	4. Booking Still Visible After Cancellation:	1. Check that the booking list refreshes appropriately after cancellation to reflect the current state. 2. Test the booking list to ensure that it updates without requiring a manual refresh.

UPDATE BOOKING (LECTURER)

TEST CASE NUMBER	7
TEST SCENARIO	**<u>Unsuccessful Booking Update Due to Validation Errors</u>** Ensure that the system handles unsuccessful booking updates correctly by validating event details and providing appropriate feedback for validation errors.
TEST STEPS	1. The lecturer logs into the Reservify system. Navigates to the "Manage Bookings" page. Selects the booking they wish to update from the list of existing bookings. 2. **Input Invalid Event Details**: The lecturer enters new event details that conflict with existing bookings. 3. **Submit Update**: The lecturer clicks the "Update" button to submit the changes.
PREREQUISITES	1. The venue booking system, Reservify, is operational and accessible without any downtime. 2. The lecturer is logged in with appropriate permissions as a lecturer. 3. At least one existing booking is present in the lecturer's account, which conflicts with the new booking details.
TEST DATA	• **Venue Category** • **Date** • **Duration** • **Capacity**
VALUES FOR TEST DATA	**Selected Booking ID**: 205 **New Details**: • **Venue Category**: Room 101 • **Date**: 2024-11-01 • **Duration**: 3 hours (overlapping with another booking) • **Capacity**: 50
EXPECTED RESULTS	1. The system identifies the validation error due to the overlapping times and displays an appropriate error message. 2. The lecturer is prompted to review their input and correct the conflicting details.

ACTUAL RESULTS	(To be filled after execution) Example result: "Error: The selected time overlaps with an existing booking. Please choose a different time or date."	
TEST STATUS	**PASS**	
	The system correctly identifies scheduling conflicts and informs the user about the validation errors.	
	FAIL	
	Type of Failure	**Solution**
	1. Validation Errors Not Displayed	1. Review the conflict detection logic to ensure that it accurately identifies overlaps and generates error messages. 2. Test that validation messages are clear and informative.
	2. System Allows Overlapping Updates	1. Ensure that the booking update process is blocked when conflicts are detected. 2. Validate that the system enforces business rules regarding overlapping bookings.
	3. Lack of Specific Feedback	1. Improve feedback messages to provide clear guidance on what needs to be corrected. 2. Confirm that the system offers specific suggestions or options for resolving conflicts.
	4. Inconsistent Validation Behaviour	1. Test various scenarios with different types of conflicts to ensure consistent validation logic. 2. Conduct tests with multiple overlapping bookings to validate the robustness of the conflict detection.

UPDATE STUDENT PROFILE (BY ADMIN)

TEST CASE NUMBER	1
TEST SCENARIO	**Access Update User Profile Interface** Admin accesses the Update User Profile interface to update student details.
TEST STEPS	1. Admin logs into the system using valid credentials. 2. Admin navigates to the "Dashboard" after successful login. 3. Admin clicks on the "User Management" tab or equivalent section. 4. Admin selects the "Update User Profile" option from the user management menu. 5. The system redirects the admin to the "Update User Profile" interface.
PREREQUISITES	1. Admin is logged in with valid credentials. 2. The "Update User Profile" feature must be enabled and available in the system.
TEST DATA	None
VALUES FOR TEST DATA	None
EXPECTED RESULTS	The system successfully loads the "Update User Profile" interface. The admin can access the update profile interface.
ACTUAL RESULTS	The system loaded the “Update User Profile” interface, and the admin was able to access the update user profile page as expected.
TEST STATUS	**PASS**
	Admin can access the update form.
	FAIL

Type of Failure	**Solution**
1.Update User Profile Page fails to load.	Check the connection between the client and the server.
2. Admin lacks the required permissions to access the update interface.	Ensure the admin role has the necessary permissions set in the user management system to access the update profile page and edit student’s profile.

UPDATE STUDENT PROFILE (BY ADMIN)

TEST CASE NUMBER	2
TEST SCENARIO	**Retrieve All Students** **Admin selects "Student" as the user type and retrieves the list of all students.**
TEST STEPS	1. Admin selects "Student" from the user type dropdown list, choosing to retrieve all student profiles in the system. 2. Admin clicks on the "Retrieve Students" button to initiate the process of loading all students. 3. The system queries the database for all users who have a "Student" role or designation. 4. The system displays a list of all students, including their basic details such as first name, last name, email, and contact number. 5. Admin can view the list and decide to either filter or proceed to update any student profile.
PREREQUISITES	✓ Admin is logged in with valid credentials and has access to the "Update User Profile" interface. ✓ The system must have student user profiles stored in the database. ✓ Admin has the necessary permissions to view and manage student profiles. ✓ The system is functioning without any connection issues to the database.
TEST DATA	User Type
VALUES FOR TEST DATA	User Type: Student
EXPECTED RESULTS	1. All student profiles in the system are retrieved and displayed without delay. 2. The list contains essential information for each student, such as: • First Name • Last Name • Email • Contact Number 3. The system ensures that only student profiles are retrieved and displayed. The list is displayed with no errors or missing information
ACTUAL RESULTS	As expected, a list of all students was displayed.
TEST STATUS	**PASS** **The system displays a list of all students.**

FAIL

Type of Failure	Solution
1. The system does not retrieve or display the student list.	Check the database connection and query used to retrieve student data. Validate that the student records exist.
2. The system displays users other than students.	Ensure the "Student" filter is applied correctly during retrieval.

UPDATE STUDENT PROFILE (BY ADMIN)

TEST CASE NUMBER	3
TEST SCENARIO	**Filter Students** Admin filters the student list based on specified criteria
TEST STEPS	1. Admin enters filter criteria such as first name, last name, email, or contact number in the respective search fields to narrow down the list of students. 2. Example filter criteria: a. First Name: "John" b. Last Name: "Doe" c. Email: "john.doe@example.com" d. Contact Number: "555-123-4567" 3. Admin clicks on the "Filter" button to apply the search criteria. 4. The system queries the database based on the entered filter criteria and returns matching student profiles. 5. The filtered list of students is displayed, containing only the profiles that match the entered criteria.
PREREQUISITES	✓ Admin is logged in with valid credentials and has access to the "Update User Profile" interface. ✓ The system must have student profiles stored in the database. ✓ The filter criteria entered by the admin must match existing student profile information. ✓ The admin has the necessary permissions to filter and view student profiles.
TEST DATA	First Name, Last Name, Email, Contact Number
VALUES FOR TEST DATA	First Name: Jack Last Name: Daniels Email: jackdaniels@gmail.com Contact Number: 0712345678
EXPECTED RESULTS	The system successfully filters the student list based on the specified criteria. The filtered list contains only students whose profiles match the entered filter criteria: ✓ First Name: Jack ✓ Last Name: Daniels ✓ Email: jackdaniels@gmail.com ✓ Contact Number: 0712345678
ACTUAL RESULTS	As expected, the filtered list contains only students whose profiles match the entered filter criteria.
TEST STATUS	**PASS** The system displays a filtered list of students based on the criteria.

	FAIL	
	Type of Failure	**Solution**
	1. The system does not apply the filter criteria and displays the entire list	Verify that the filtering logic is correctly implemented and that the input values are being passed to the filter function.
	2. The system fails to show results, even though matching records exist.	Review the query logic to ensure proper matching criteria are used for the filter.

UPDATE STUDENT PROFILE (BY ADMIN)

TEST CASE NUMBER	4
TEST SCENARIO	**Select User to Update** Admin selects a user from the filtered list to update their profile.
TEST STEPS	1. Admin accesses the Update User Profile interface. 2. Admin selects "Student" as the user type to display the list of all students. 3. Admin enters filter criteria (e.g., first name, last name, email) to narrow down the student list. 4. The filtered list of students is displayed based on the criteria entered. 5. Admin selects a specific student from the displayed list. 1. The system retrieves and displays the current profile details of the selected student.
PREREQUISITES	1. Admin is logged in and has access to the Update User Profile interface. 2. There are existing students in the system that match the filter criteria.
TEST DATA	Filter Criteria: ✓ First Name: "Alice" ✓ Last Name: "Johnson" ✓ Email: "alice.johnson@example.com"
VALUES FOR TEST DATA	Current Profile Data of Selected Student: ✓ First Name: "Alice" ✓ Last Name: "Johnson" ✓ Email: "alice.johnson@example.com" ✓ Contact Number: "555-123-4567"
EXPECTED RESULTS	1. The filtered list of students displays the correct entries based on the filter criteria. 2. The admin can select the desired student from the filtered list. 3. The system successfully retrieves and displays the current profile details for the selected student:

<table>
<tr><td></td><td>✓ First Name: "Alice"
✓ Last Name: "Johnson"
✓ Email: "alice.johnson@example.com"
✓ Contact Number: "555-123-4567"</td></tr>
<tr><td>ACTUAL RESULTS</td><td>1. The filtered list displays the correct students, including "Alice Johnson."
2. Upon selecting "Alice Johnson," the current profile details are displayed as expected:
✓ First Name: "Alice"
✓ Last Name: "Johnson"
✓ Email: "alice.johnson@example.com"
✓ Contact Number: "555-123-4567"</td></tr>
<tr><td rowspan="5">TEST STATUS</td><td colspan="2">PASS</td></tr>
<tr><td colspan="2">The admin successfully selected the user from the filtered list, and the current profile details were displayed correctly.</td></tr>
<tr><td colspan="2">FAIL</td></tr>
<tr><td>Type of Failure</td><td>Solution</td></tr>
<tr><td>1. The filtered list does not show the correct students.</td><td>1.1 Review the filter logic to ensure it is functioning correctly.
1.2 Ensure that the student records in the database match the filter criteria.</td></tr>
<tr><td>2. The system fails to retrieve the selected student’s profile.</td><td>2.1 Check the connection to the database to ensure it is operational.
2.2 Verify that the student ID or identifier being used is valid.</td></tr>
</table>

UPDATE STUDENT PROFILE (BY ADMIN)

TEST CASE NUMBER	5
TEST SCENARIO	**Edit Profile Fields** Admin edits the profile fields of the selected user, To edit the profile fields of a selected student user.
TEST STEPS	1. Admin accesses the Update User Profile interface. 2. Admin selects "Student" as the user type to display the list of all students. 3. Admin filters the list of students (if needed) to find a specific student. 4. Admin selects a student user from the displayed list. 5. The current profile details of the selected student are displayed. 6. Admin updates the following profile fields 6.1 First Name: Change from "Jane" to "John" 6.2 Last Name: Change from "Smith" to "Doe" 6.3 Email: Change from "jane.smith@example.com" to john.doe@example.com 6.4 Contact Number: Change from "987-654-3210" to "123-456-7890" 7. Admin submits the updated information.
PREREQUISITES	1. Admin is logged in and has access to the Update User Profile interface. 2. The student user selected for editing must exist in the system.
TEST DATA	1. First Name 2. Last Name 3. Email 4. Contact Number
VALUES FOR TEST DATA	**Current Profile Data:** • First Name: "Jane" • Last Name: "Smith" • Email: "jane.smith@example.com" • Contact Number: "987-654-3210" **Updated Profile Data:** • First Name: "John" • Last Name: "Doe" • Email: "john.doe@example.com" • Contact Number: "123-456-7890"
EXPECTED RESULTS	1. The system successfully validates the updated information. 2. Admin is prompted to confirm the changes.

<table>
<tr><td></td><td>3. Upon confirmation, the profile updates are applied.
4. A success message, "Profile updated successfully!" is displayed to the admin.
The updated profile details are correctly reflected in the user profile interface:

➢ First Name: "John"
➢ Last Name: "Doe"
➢ Email: "john.doe@example.com"
➢ Contact Number: "123-456-7890"

5. The updated profile information is saved in the database without errors.
6. The admin can retrieve the updated profile from the user list, confirming the changes.</td></tr>
<tr><td>ACTUAL RESULTS</td><td>1. Example result: "Profile updated successfully!"
2. Updated profile details displayed: First Name: "John", Last Name: "Doe", Email: "john.doe@example.com", Contact Number: "123-456-7890".</td></tr>
<tr><td rowspan="5">TEST STATUS</td><td>PASS</td></tr>
<tr><td>The admin successfully edited the profile fields of the selected student user, and the changes were reflected in the system as expected.</td></tr>
<tr><td>FAIL</td></tr>
<tr><td>Type of Failure</td><td>Solution</td></tr>
<tr><td>1. Validation errors are displayed.</td><td>1.1 Review the input values for any errors or constraints (e.g., email format).
1.2 Ensure that the email address is unique in the database.</td></tr>
<tr><td>2. The profile is not updated.</td><td>2.1 Check system logs for any backend processing issues during the update.
2.2 Confirm that the admin has the necessary permissions to perform the update.</td></tr>
</table>

UPDATE STUDENT PROFILE (BY ADMIN)

TEST CASE NUMBER	6
TEST SCENARIO	**Confirm Profile Update** Admin confirms the update after successful validation of the changes.
TEST STEPS	1. Admin accesses the Update User Profile interface. 2. Admin selects a student user to edit. 3. Admin updates the necessary profile fields. 4. Admin submits the updated information. 5. The system validates the updated information. 6. Admin receives a prompt to confirm the changes. 7. Admin clicks on the "Confirm Update" button.
PREREQUISITES	1. Admin is logged in and has access to the Update User Profile interface. 2. The student user selected for editing must exist in the system. 3. The updated information must pass all validation checks.
TEST DATA	✓ **First Name** ✓ **Last Name** ✓ **Email** ✓ **Contact Number**
VALUES FOR TEST DATA	**Updated Profile Data:** • First Name: "Alice" • Last Name: "Johnson" • Email: "alice.johnson@example.com" • Contact Number: "555-123-4567"
EXPECTED RESULTS	1. The system successfully validates the updated information. 2. Admin is prompted to confirm the changes with the updated details displayed. 3. Upon clicking "Confirm Update," the profile updates are applied. 4. A success message, "Profile updated successfully!" is displayed to the admin. 5. The updated profile details are reflected in the user profile interface: ✓ First Name: "Alice" ✓ Last Name: "Johnson" ✓ Email: "alice.johnson@example.com" ✓ Contact Number: "555-123-4567" 6. The updated profile information is saved in the database without errors. 7. The admin can retrieve the updated profile from the user list, confirming the changes.

ACTUAL RESULTS	1. Example result: "Profile updated successfully!" 2. Updated profile details displayed:
TEST STATUS	**PASS**
	The admin successfully confirmed the profile update, and the changes were reflected in the system as expected.
	FAIL

Type of Failure	Solution
1. Admin does not receive a confirmation prompt	1.1 Verify that the validation process is working correctly. 1.2 Check for any system errors or logs that might indicate a failure in the prompt display.
2. The profile is not updated after confirmation	2.1 Check system logs for any backend processing issues during the update. 2.2 Confirm that the admin has the necessary permissions to perform the update.
3. The profile update is not reflected in the user interface.	3.1 Refresh the user interface to ensure it displays the latest data. 3.2 Verify that the update was successfully committed to the database.

UPDATE STUDENT PROFILE (BY ADMIN)

TEST CASE NUMBER	7
TEST SCENARIO	**Cancel Profile Update** Admin cancels the profile update process.
TEST STEPS	1. Admin accesses the Update User Profile interface. 2. Admin selects a student user to edit. 3. Admin updates the necessary profile fields. 4. Admin submits the updated information. 5. The system validates the updated information. 6. Admin receives a prompt to confirm the changes. 7. Admin clicks on the "Cancel Update" button instead of confirming.

<table>
<tr><td>PREREQUISITES</td><td colspan="2">1. Admin is logged in and has access to the Update User Profile interface.
2. The student user selected for editing must exist in the system.
3. The updated information must be displayed for confirmation.</td></tr>
<tr><td>TEST DATA</td><td colspan="2">✓ First Name
✓ Last Name
✓ Email
✓ Contact Number</td></tr>
<tr><td>VALUES FOR TEST DATA</td><td colspan="2">Updated Profile Data:
• First Name: "Bob"
• Last Name: "Williams"
• Email: "bob.williams@example.com"
• Contact Number: "555-987-6543"</td></tr>
<tr><td>EXPECTED RESULTS</td><td colspan="2">1. The system successfully validates the updated information.
2. Admin is prompted to confirm the changes with the updated details displayed.
3. Upon clicking "Cancel Update," the system does not apply any changes.
4. A cancellation message, "Profile update cancelled." is displayed to the admin.
5. The user profile interface remains unchanged, showing the original profile details:
✓ First Name: "Bob" (original)
✓ Last Name: "Williams" (original)
✓ Email: "bob.williams@example.com" (original)
✓ Contact Number: "555-987-6543" (original)
6. No changes are saved in the database.</td></tr>
<tr><td>ACTUAL RESULTS</td><td colspan="2">1. Example result: "Profile update cancelled."
2. Original profile details displayed remain unchanged:
• First Name: "Bob"
• Last Name: "Williams"
• Email: "bob.williams@example.com"
• Contact Number: "555-987-6543".</td></tr>
<tr><td rowspan="4">TEST STATUS</td><td colspan="2">PASS</td></tr>
<tr><td colspan="2">The admin successfully cancelled the profile update process, and no changes were made to the user profile.</td></tr>
<tr><td colspan="2">FAIL</td></tr>
<tr><td>Type of Failure</td><td>Solution</td></tr>
<tr><td>1. Admin does not receive a cancellation message</td><td>1.1 Verify that the cancellation process is functioning correctly.
1.2 Check for any system errors or logs that might indicate a failure in the message display.</td></tr>
</table>

	2. The profile is updated despite cancellation	2.1 Check system logs for any backend processing issues during the cancel operation. 2.2 Ensure that the cancellation logic is correctly implemented in the system.
	3. System does not cancel the update as expected.	3.1 Check the cancel operation's logic in the backend to ensure it is executed properly. 3.2 Ensure that the admin's cancellation action is being recorded correctly.

UPDATE STUDENT PROFILE (BY ADMIN)

TEST CASE NUMBER	8
TEST SCENARIO	**Validation of Input Fields** Admin submits invalid input data and checks for validation errors.
TEST STEPS	1. Admin accesses the Update User Profile interface. 2. Admin selects a student user to edit. 3. Admin attempts to update the following profile fields with invalid data: ✓ First Name: Leave blank ✓ Last Name: "Doe123" (contains numbers) ✓ Email: "invalid-email-format" (incorrect email format) ✓ Contact Number: "abc-def-ghij" (non-numeric format) 4. Admin submits the updated information. 5. The system validates the input data.
PREREQUISITES	7. Admin is logged in and has access to the Update User Profile interface. 8. The student user selected for editing must exist in the system.
TEST DATA	✓ **First Name** ✓ **Last Name** ✓ **Email** ✓ **Contact Number**
VALUES FOR TEST DATA	✓ **First Name:** "" ✓ **Last Name:** "Doe456" ✓ **Email:** "wrong.email@format" ✓ **Contact Number:** "12A-345-6789"
EXPECTED RESULTS	1. The system identifies that the First Name field is required and displays a validation error: "First Name cannot be empty."

	2. The system identifies that the Last Name field contains invalid characters and displays a validation error: "Last Name must not contain numbers." 3. The system identifies that the Email format is invalid and displays a validation error: "Please enter a valid email address." 4. The system identifies that the Contact Number format is incorrect and displays a validation error: "Contact Number must be numeric." 5. Admin is prompted to correct the input fields before resubmitting the information. 6. No changes are made to the user profile.
ACTUAL RESULTS	1. Example result: "First Name cannot be empty." 2. Example result: "Last Name must not contain numbers." 3. Example result: "Please enter a valid email address." 4. Example result: "Contact Number must be numeric."
TEST STATUS	**PASS**
	The system successfully identifies all validation errors and prompts the admin to correct the input data.
	FAIL

Type of Failure	Solution
1. Validation errors are not displayed for certain fields.	1.1 Review the input validation logic to ensure all fields are checked. 1.2 Check for any front-end or back-end validation issues.
2. Changes are incorrectly applied despite invalid input.	2.1 Check the update process to ensure it only proceeds with valid input. 2.2 Ensure that all input validation rules are enforced correctly.
3. Incorrect validation messages are shown.	3.1 Review the input validation rules to ensure they are accurately implemented. 3.2 Test with various inputs to ensure the validation logic works as intended.

UPDATE STUDENT PROFILE (BY ADMIN)

TEST CASE NUMBER	9
TEST SCENARIO	**Finalize Profile Update** Admin finalizes the profile update process and checks for completion confirmation.
TEST STEPS	1. Admin accesses the Update User Profile interface. 2. Admin selects a student user and edits the profile fields with valid data. 3. Admin confirms the updates after successful validation. 4. Admin clicks on the "Finalize Update" button. 5. The system processes the finalization of the profile update.

PREREQUISITES	1. Admin is logged in and has access to the Update User Profile interface. 2. The student user selected for editing must exist in the system and have valid updates made.
TEST DATA	1. **User ID:** Unique identifier of the student user being updated. 2. **Updated Profile Data:** • First Name: "John" • Last Name: "Doe" • Email: "john.doe@example.com" • Contact Number: "123-456-7890"
VALUES FOR TEST DATA	✓ **User ID:** 101 ✓ **Updated First Name:** "John" ✓ **Updated Last Name:** "Doe" ✓ **Updated Email:** "john.doe@example.com" ✓ **Updated Contact Number:** "123-456-7890"
EXPECTED RESULTS	1. The system successfully processes the finalization of the profile update. 2. A confirmation message is displayed: "Profile update finalized successfully!" 3. The updated profile details are correctly reflected in the user profile interface. 4. The updated information is saved in the database without errors. 5. The admin can retrieve the finalized profile from the user list, confirming the changes.
ACTUAL RESULTS	1. Example result: "Profile update finalized successfully!" 2. Updated profile details displayed: First Name: "John", Last Name: "Doe", Email: "john.doe@example.com", Contact Number: "123-456-7890".
TEST STATUS	**PASS**
	The admin successfully finalizes the profile update, and a confirmation message is displayed as expected.
	FAIL

Type of Failure	Solution
1. Confirmation message is not displayed	1.1 Check system logs for any errors during the finalization process. 1.2 Ensure the front-end is correctly receiving and displaying the confirmation.
2. Updated profile details are not saved correctly.	2.1 Review database transaction handling to ensure changes are committed. 2.2 Confirm that the admin has the necessary permissions to perform the finalization.
3. The finalization process takes too long or times out	3.1 Check server performance and optimize database queries for efficiency. 3.2 Implement appropriate timeout handling and user feedback during the process.

8. PEER REVIEW FEEDBACK

INFO2001A - PEER EVALUATION

Team No: **32** Team Name: **TECHFUSION** Date: **18/10/2024**

Each team is required to complete a peer evaluation. The following aspects of the team's work should be taken into account when deciding on the percentage contribution to allocate to each member:

- Quality of individual contribution
- Timely submission of work
- Online Availability and communication

The following principles apply:

1. Every team member must agree on percentage contributions and sign the form.
2. The evaluation is not anonymous, but an adult assessment of contribution.
3. Think carefully about your assessment of your peers. Try to avoid extreme assessments unless they are fully justified.
4. In a team of five (5), if each member contributed equally, then each would be allocated 20%, in a team of six (6) each would be allocated 16.67%.
5. A team member who scores higher than the average will be awarded a project mark that is proportionally above the mark obtained by the team.
6. A team member who scores lower than the average will be awarded a project mark that is proportionally below the mark obtained by the team.
7. Extreme cases, e.g. where a student has made no contribution at all, will be handled on a case by case basis by the project coordinator (lecturer)

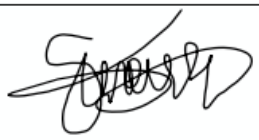
Member	Member Name	Contribution (%)
1	Arno Strauss	35.00
2	Dintle Maine	33.50
3	Khulekani Mtshali	31.50
4		
5		
6		
Total		**100%**

Team Signatures

Member	Member Signatures
1	A. Strauss
2	D. Maine
3	K. Mtshali
4	
5	
6	

Motivation (where there are issues that need explanation):

9. TECHFUSION TEAM SIGN-OFF

Arno Strauss	
Dintle Maine	
Khulekani Mtshali	

WITS UNIVERSITY

SBS SCHOOL OF BUSINESS SCIENCES

RESERVIFY® Implementation Phase

Milestone 3:

Sequence diagrams + Use Case Realization + UI-UX/Report design

Group Number: 32

Team Name: TechFusion

TechFusion Partner Details

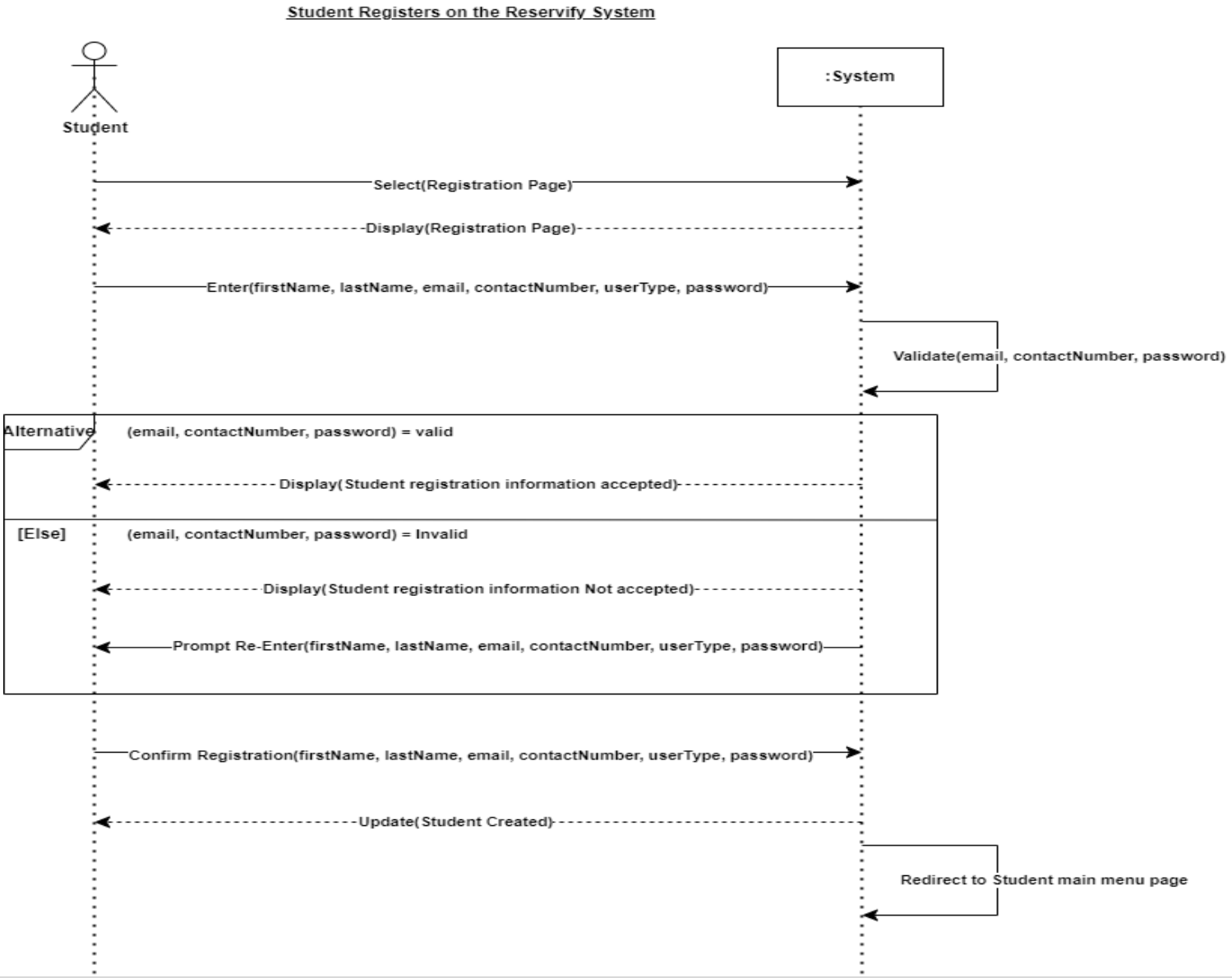
Member:	Name:	Surname:	Student Number:
1	Arno	Strauss	2613224
2	Dintle	Maine	2451837
3	Khulekani	Mtshali	2550411

TechFusion Logo

TABLE OF CONTENTS

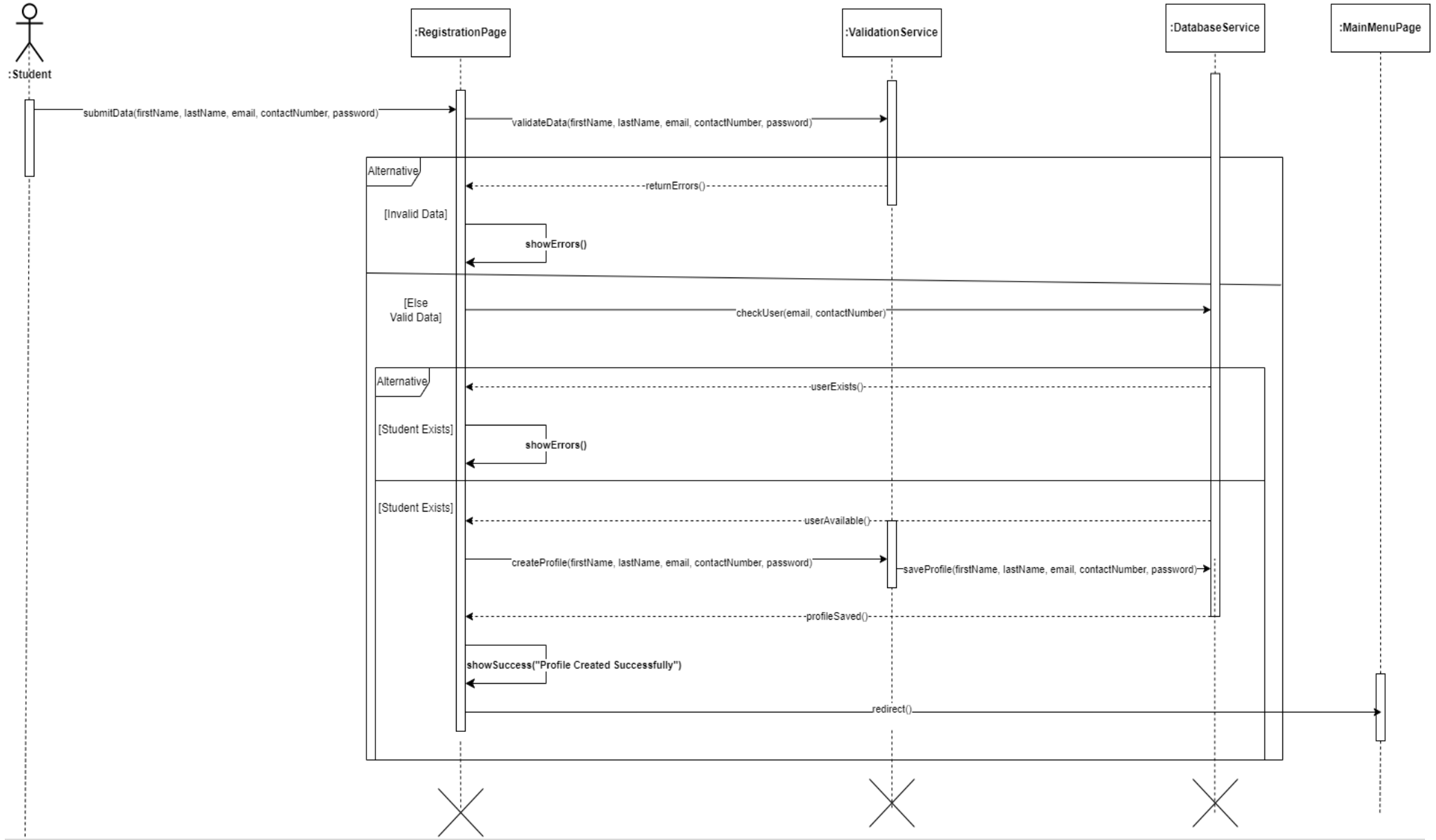
1. System Sequence Diagrams (SSD)

Create Profile (Student)

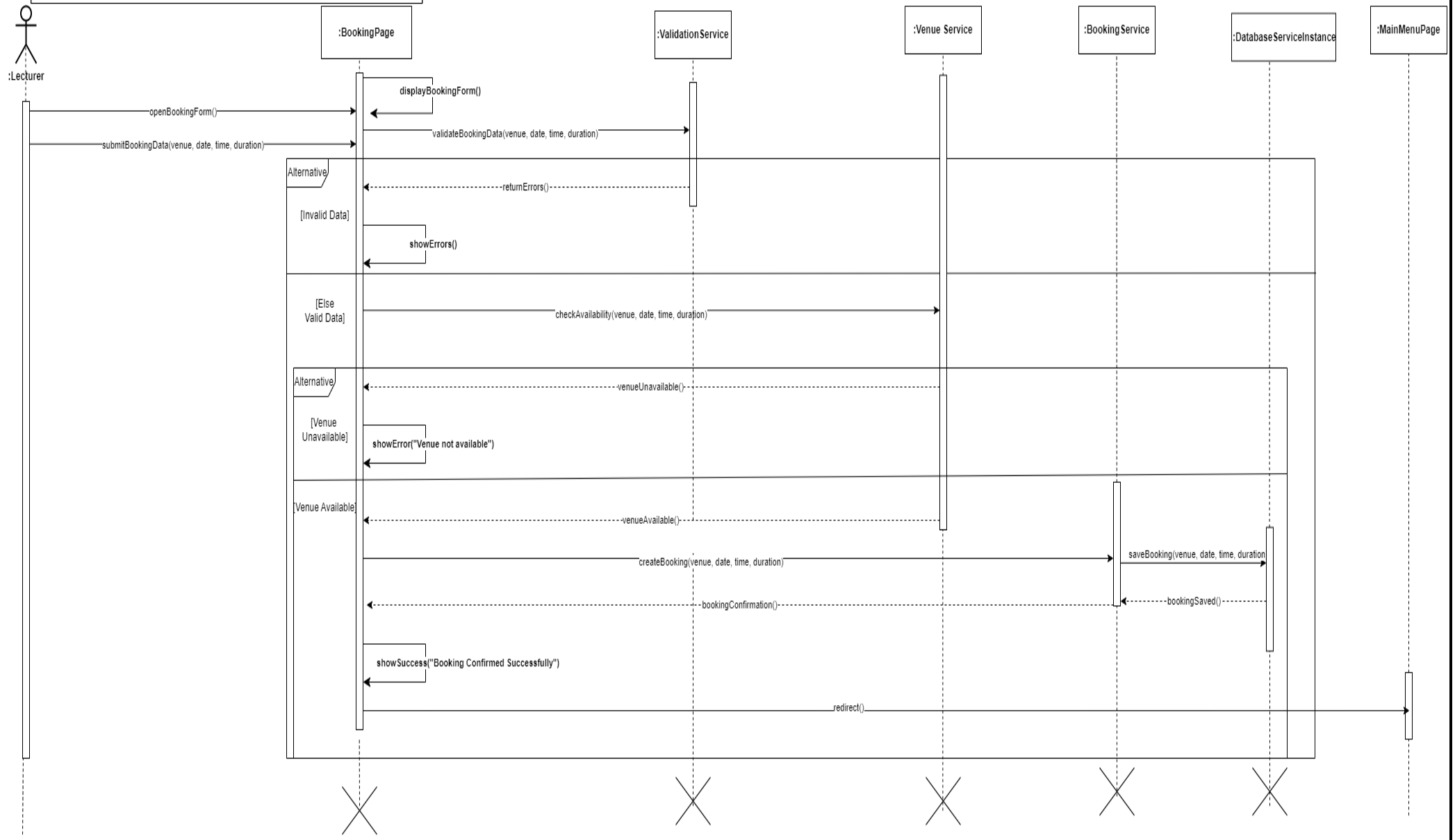


2. SEQUENCE DIAGRAMS

Create Profile (Student)

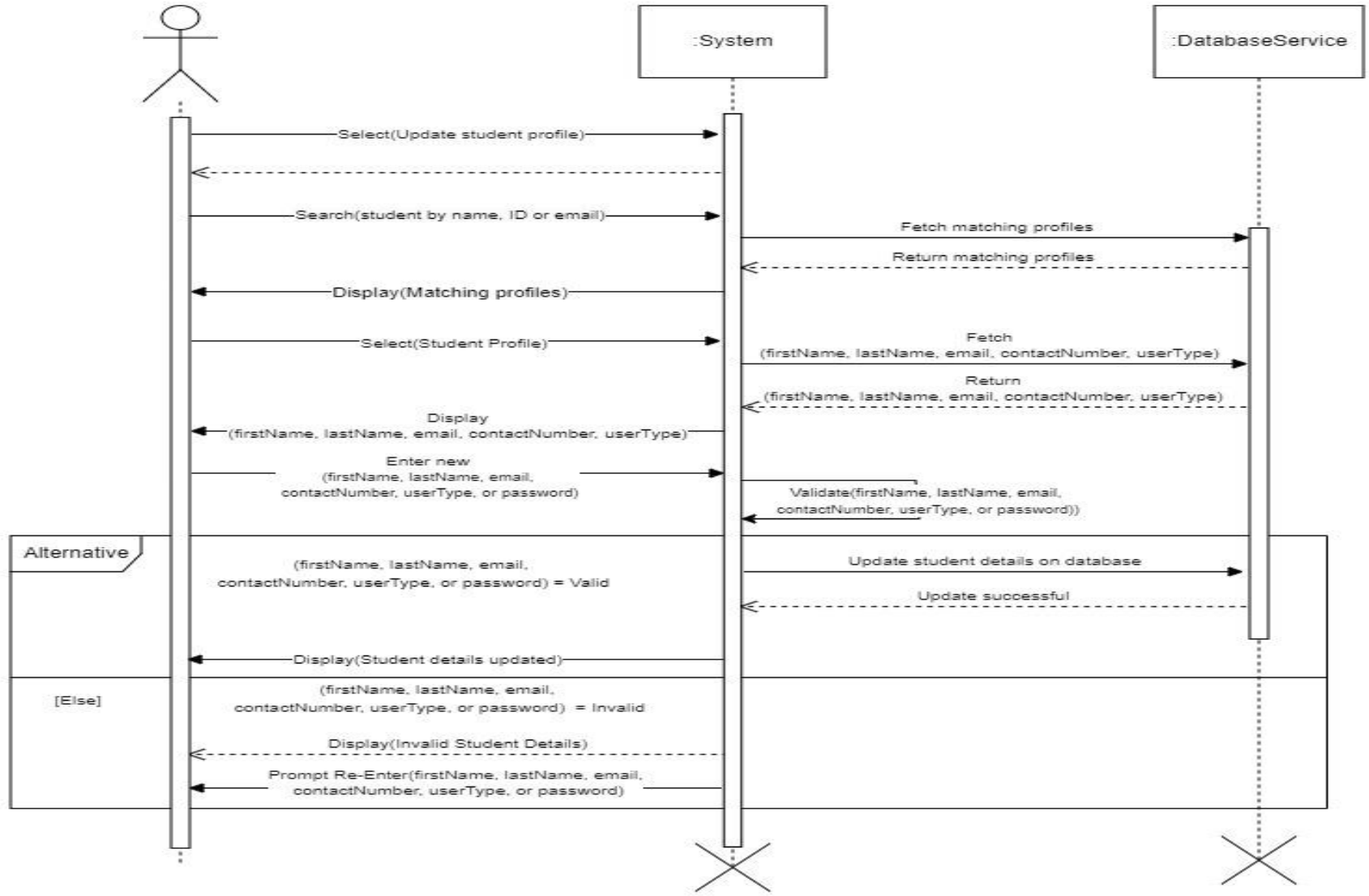


To make a venue booking (lecturer)



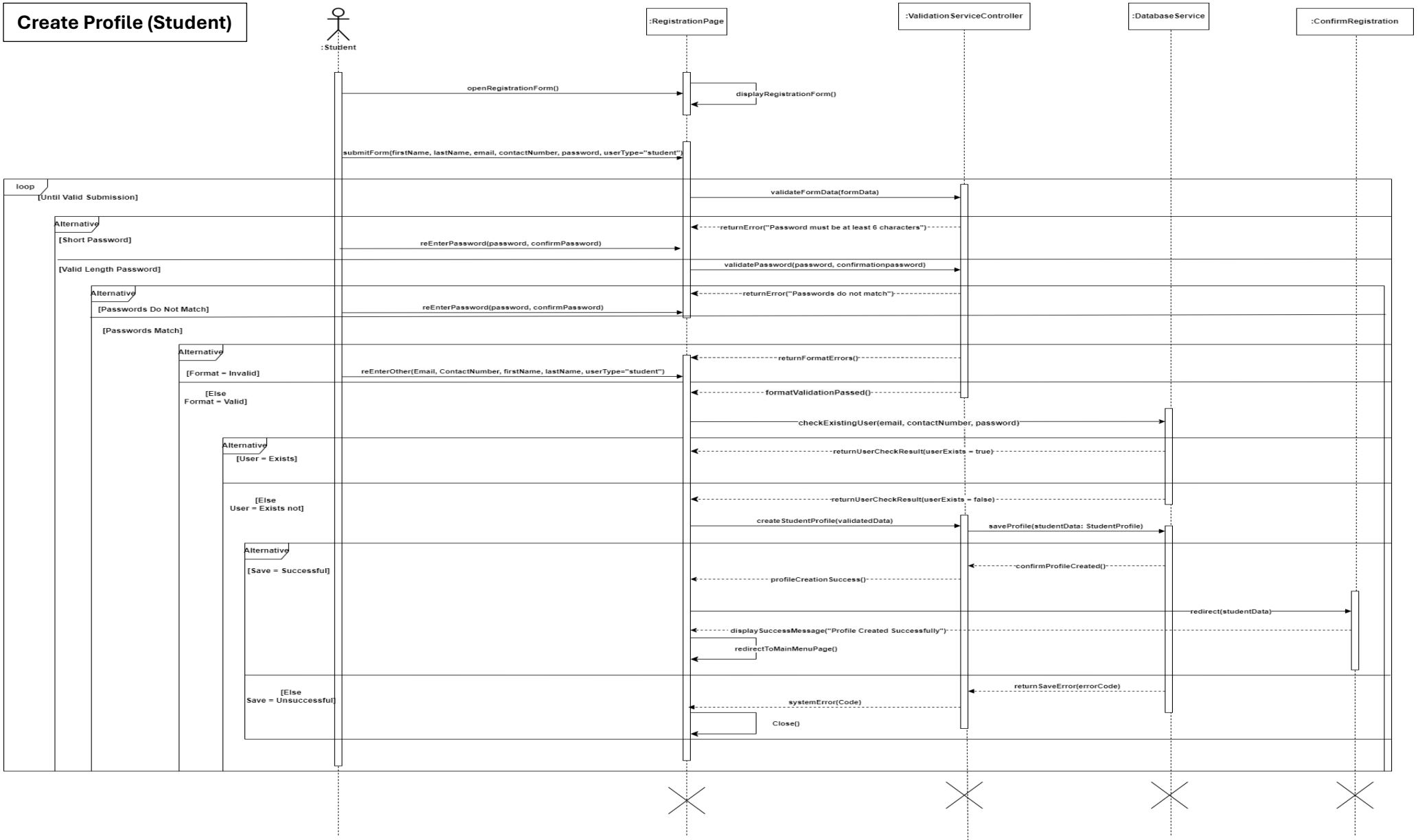
Admin Updates Student Profile On Reversify

To update profile (student – admin)

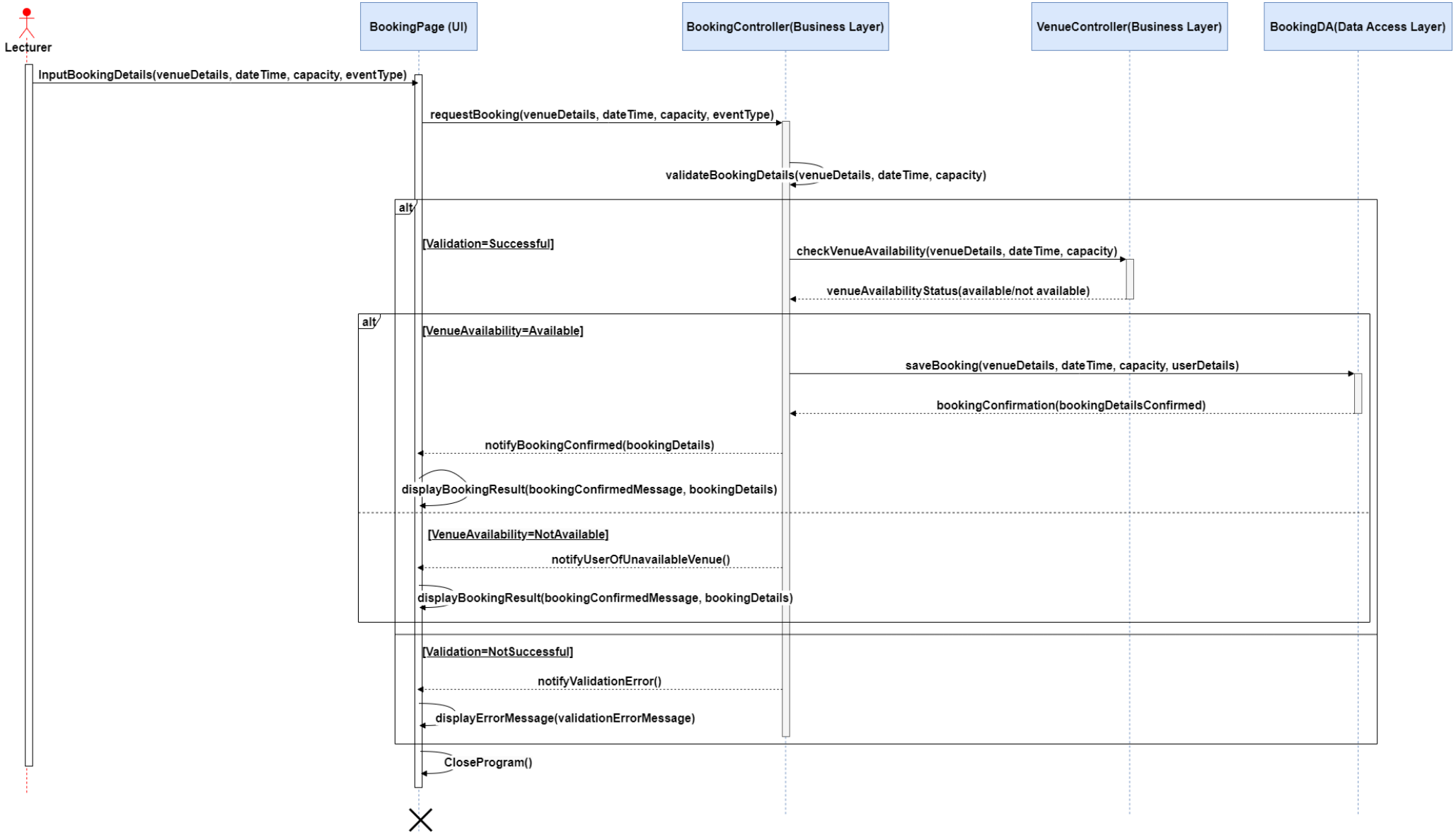


3. USE CASE REALIZATION

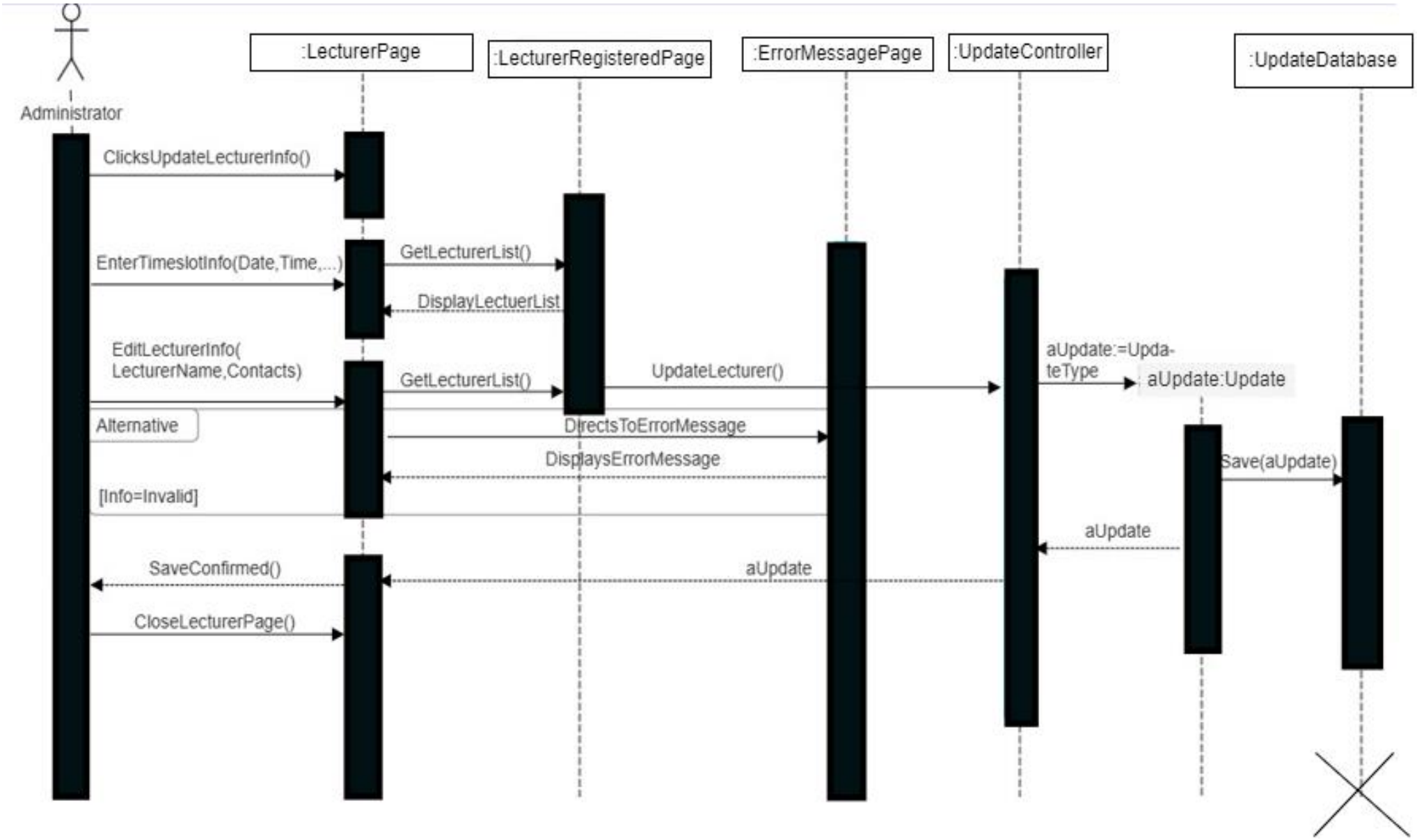
Create Profile (Student)



Make a venue booking (Lecturer)



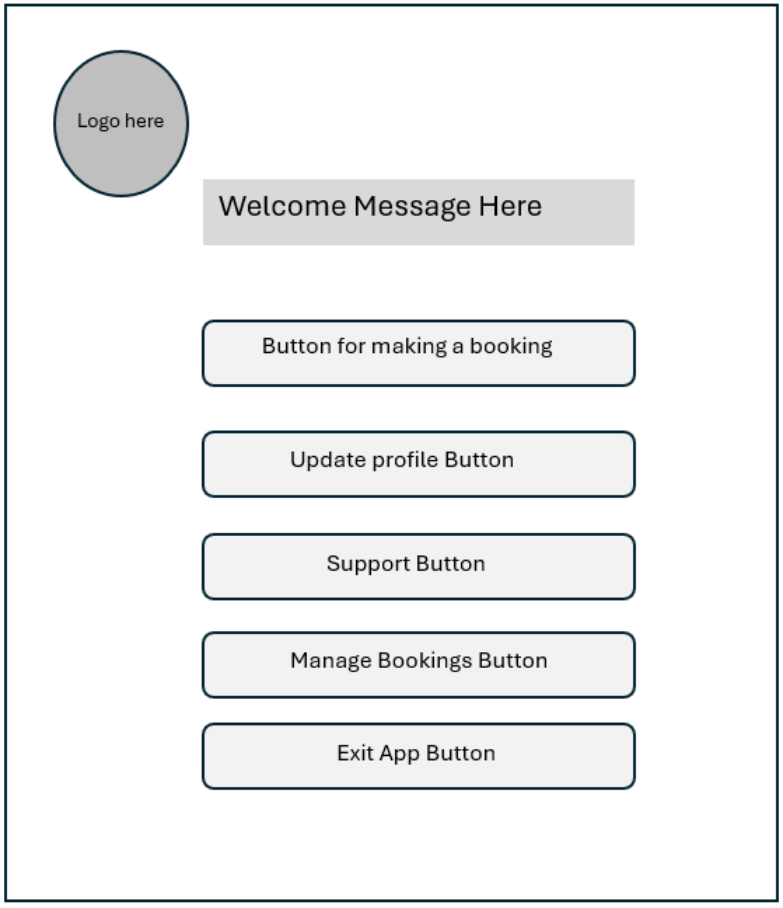
Update student profile (Admin)



4. WIREFRAME

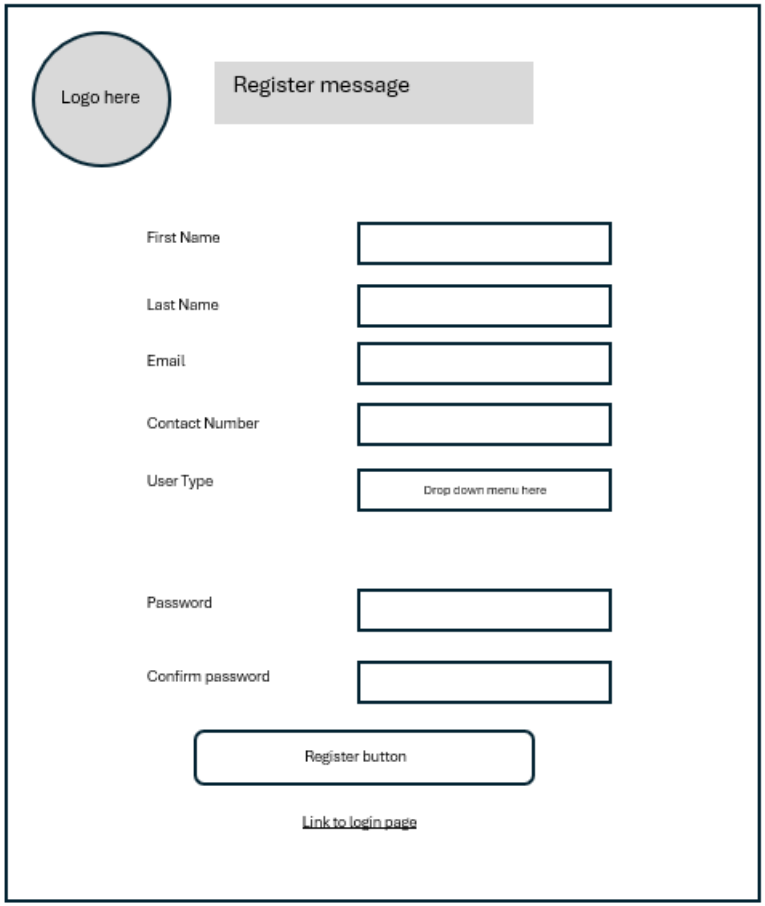
Main menu from Student view

Menu Page



Purpose of the Page:	The main menu serves as the central hub for users after logging in, providing access to various functions within the system tailored to their specific roles in this case the student.
Elements:	**Make Booking Button:** Redirects users to a dedicated page for making venue bookings. **Manage Bookings Button:** Navigates users to a page where they can view, edit, or cancel their existing bookings. **Profile Button:** Redirects users to their profile page, allowing them to view and update their personal details. **Support Button:** Opens a page for users to access help and support resources. **Logout Button:** Logs users out of the system and redirects them to the login page.
Navigation:	This page functions as a dashboard, featuring clearly labelled buttons for each function. Each option is designed for quick access, ensuring minimal navigation effort for users.
Creative goals:	Create a simple, intuitive interface with large, clickable buttons that make navigation clear and easy for users.

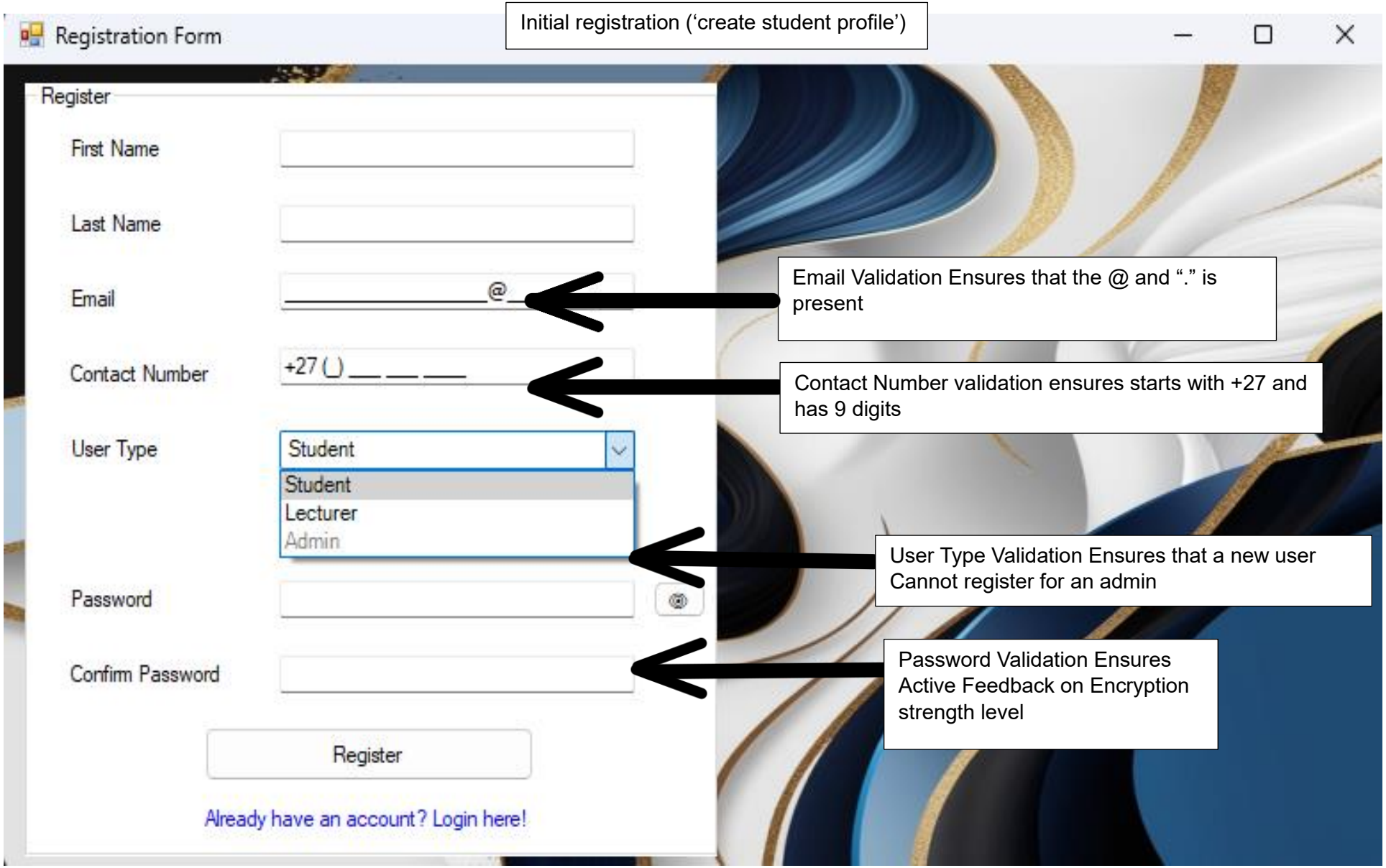
Student registration page



Purpose of the Page:	This registration page allows new users to create an account within the venue booking system, ensuring proper registration for access to its features.
Elements:	**First Name, Last Name, Email, and Contact Number Fields:** Input fields for collecting essential personal information. **User Type Drop-Down Menu:** Enables users to select their role (student, lecturer, or admin) to tailor their experience. **Password and Confirm Password Fields:** Secure fields for password creation, with validation to ensure both entries match. **Register Button:** Submits the registration form when clicked. **Navigation Link:** Provides a link for users with existing accounts to easily return to the login page.
Navigation:	The layout is designed to facilitate intuitive navigation, ensuring users can easily find and complete necessary actions. Clear labels and logical flow guide users through the registration process.
Creative goals:	The wireframe should embody a modern and user-friendly design, appealing to the target audience with a clean and engaging layout.

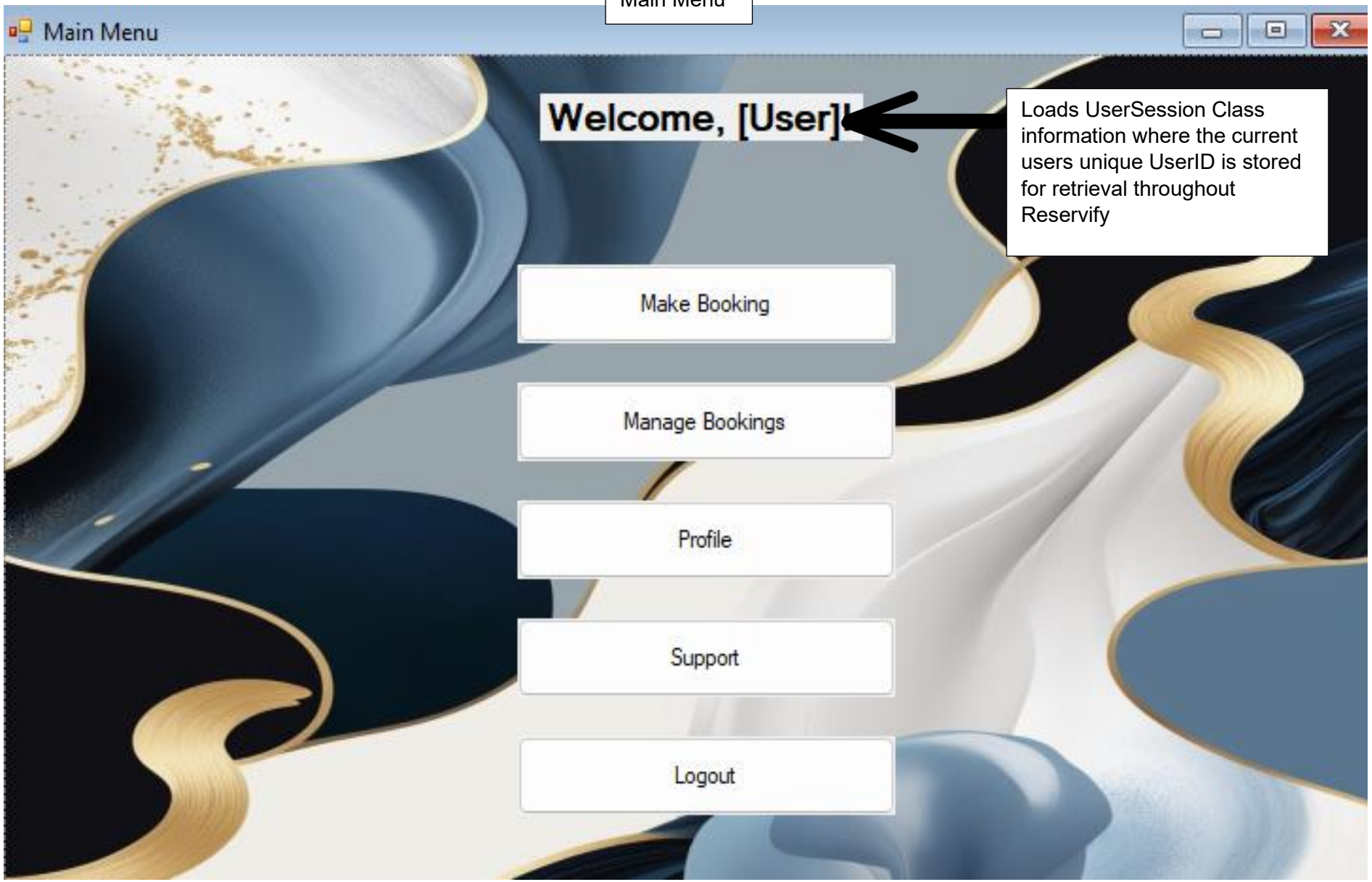
5. USER INTERFACE DESIGN (IMAGES OF THE C# FORM)

Initial registration ('create student profile')



Problem space	**User Needs**: The student needs to register by entering personal details such as name, email, and contact number, select user type, as well as create login credentials. The interface should guide them to fill out these fields correctly. **Goals:** Ensure students can register quickly and without mistakes, providing real-time feedback where necessary to avoid errors **Limitations & Context:** This design is tailored for students who may not have advanced computer skills, ensuring simple and easy-to-use input fields with clear guidance on mandatory fields.
Information Architecture	**Visual Hierarchy**: 'Register' appears at the top, followed by grouped sections: personal information, contact details, and credentials. The 'Submit' button is clearly visible at the bottom. **Grouping/Placement**: Input fields are grouped by function: personal details appear first, followed by user type and login credentials. This logical progression helps users enter their information efficiently.
UI Design	**Findability**: Each input field is clearly labelled (e.g., 'First Name,' 'Email Address'), and buttons like 'Register' indicate primary actions. **Aesthetic Appeal**: The design features ample white space, a clean layout, and consistent font sizes, making the form easy to read and visually appealing. **Usability**: The form uses simple, familiar input fields, avoiding overly complex elements that could confuse the user. Instructions for invalid input are displayed in real-time to guide the user.
Design Patterns	**Component Consistency**: All buttons follow the same style (size, colour, and font), and input fields use a consistent layout with labels placed directly above the corresponding field. **Common Patterns**: The design follows common form patterns: required fields are marked with asterisks, and input validation is done via real-time feedback with inline error messages.
UI guides	**Navigation & Guidance**: The design clearly directs the user from one step to the next with appropriately placed buttons and instructions. For instance, upon filling out all required fields, the 'Submit' button becomes active, signalling that the user can proceed.
Field validation	**Validation Features**: The form implements real-time validation for key inputs such as email format and password strength. If an invalid email is entered, a red error message appears immediately below the field.

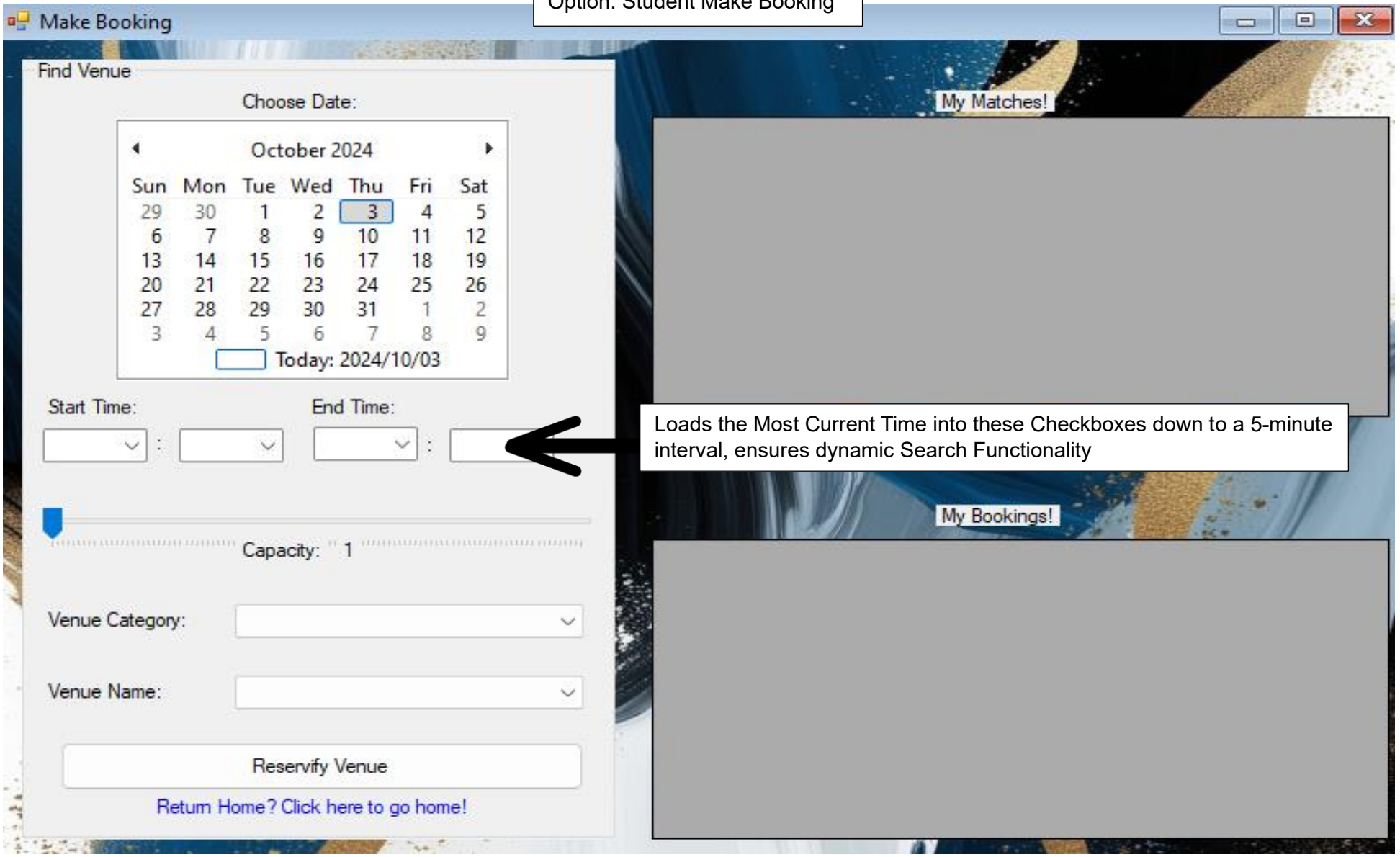
Main Menu



Loads UserSession Class information where the current users unique UserID is stored for retrieval throughout Reservify

Problem space	**User Needs**: Students need easy access to key actions, including making bookings, managing their bookings, viewing their profile, and accessing support. **Goals:** Provide a simple and clear menu with labelled buttons for easy navigation, allowing students to carry out tasks without confusion. **Limitations & Context:** Designed for students using a desktop/laptop interface, focusing on simplicity.
Information Architecture	**Visual Hierarchy**: The main menu options (e.g., 'Make a Booking,' 'Manage Bookings') are prominently displayed in large buttons, organized vertically for easy access. The most important actions are at the top. **Grouping/Placement**: The main menu options (e.g., 'Make a Booking,' 'Manage Bookings') are prominently displayed in large buttons, organized vertically for easy access. The most important actions are at the top.
UI Design	**Findability**: The menu items are well-labelled and spaced out, making each option easy to find. Icons next to the buttons could improve visual cues. **Aesthetic Appeal**: The menu is simple with large, well-spaced buttons, a clean background, and consistent fonts. **Usability**: The large buttons and clear labelling ensure ease of use, reducing the chance of user confusion.
Design Patterns	**Component Consistency**: All buttons follow the same size, style, and alignment. **Common Patterns**: Standard main menu patterns are followed with clearly labelled options for tasks like booking or accessing support.
UI guides	**Navigation & Guidance**: The main menu is easy to navigate, directing the user to where they need to go based on clearly labelled buttons.
Field validation	No input validation required at this stage.

Option: Student Make Booking



Loads the Most Current Time into these Checkboxes down to a 5-minute interval, ensures dynamic Search Functionality

Problem space	**User Needs**: The student needs to select a date, venue, and time to make a booking. The form should be simple, with easy-to-use fields and dropdowns for venue selection and time slot options. **Goals:** Ensure that students can make bookings efficiently with clear input fields and dropdown selections, minimizing confusion. **Limitations & Context:** The booking process assumes the student is familiar with the available venue options but should still include clear choices and options.
Information Architecture	**Visual Hierarchy**: Key booking fields (Date, Time, Venue) are prominently displayed at the top, followed by the 'Submit' button for booking confirmation at the bottom. **Grouping/Placement**: Fields are logically grouped: first date selection, then time and venue options, making it easy for users to follow the flow.
UI Design	**Findability**: Each field is clearly labelled, and dropdowns make venue and time slot selection straightforward. **Aesthetic Appeal**: The booking form maintains a clean and uncluttered design, with ample space between fields to avoid overwhelming the user. **Usability**: Dropdowns and calendars ensure users can easily pick their options without errors, while any invalid selections trigger immediate feedback.
Design Patterns	**Component Consistency**: All fields and dropdowns follow the same design pattern with consistent spacing and styling. **Common Patterns**: Use of calendars for date selection and dropdowns for venue and time, aligning with common booking system practices
UI guides	**Navigation & Guidance**: Clear flow from date selection to time and venue, with immediate guidance on incorrect inputs or unavailable time slots.
Field validation	**Validation Features:** Real-time validation ensures valid dates and times are chosen, and unavailable slots trigger error messages.

6. TEAM SIGN-OFF

INFO2001A - PEER EVALUATION

Team No: **32** Team Name: **TECHFUSION** Date: **03/10/2024**

Each team is required to complete a peer evaluation. The following aspects of the team's work should be taken into account when deciding on the percentage contribution to allocate to each member:

- Quality of individual contribution
- Timely submission of work
- Online Availability and communication

The following principles apply:

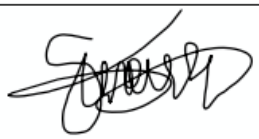
1. Every team member must agree on percentage contributions and sign the form.
2. The evaluation is not anonymous, but an adult assessment of contribution.
3. Think carefully about your assessment of your peers. Try to avoid extreme assessments unless they are fully justified.
4. In a team of five (5), if each member contributed equally, then each would be allocated 20%, in a team of six (6) each would be allocated 16.67%.
5. A team member who scores higher than the average will be awarded a project mark that is proportionally above the mark obtained by the team.
6. A team member who scores lower than the average will be awarded a project mark that is proportionally below the mark obtained by the team.
7. Extreme cases, e.g. where a student has made no contribution at all, will be handled on a case by case basis by the project coordinator (lecturer)

Member	Member Name	Contribution (%)
1	Arno Strauss	42.5
2	Dintle Maine	42.5
3	Khulekani Mtshali	15
4		
5		
6		
Total		**100%**

Team Signatures

Member	Member Signatures
1	A. Strauss
2	D. Maine
3	K. Mtshali
4	
5	
6	

Motivation (where there are issues that need explanation):

Arno Strauss	
Dintle Maine	
Khulekani Mtshali	

WITS UNIVERSITY

SCHOOL OF BUSINESS SCIENCES

RESERVIFY® Design Phase

Milestone 2: Database Design and Modelling Deliverables

Group Number: 32

Team Name: TechFusion

TechFusion Partner Details

Member:	Name:	Surname:	Student Number:
1	Arno	Strauss	2613224
2	Dintle	Maine	2451837
3	Khulekani	Mtshali	2550411

TechFusion Logo

TABLE OF CONTENTS

1. NORMALIZATION

Introduction

Normalization is a database design technique that aims to reduce redundancy and improve data integrity by organizing tables and relationships in a way that minimizes duplicate data. This process involves decomposing tables into smaller, related tables and ensuring that the relationships between these tables are well-defined. The normalization process is typically divided into several stages, known as normal forms. The key normal forms are:

First Normal Form (1NF)
Second Normal Form (2NF)
Third Normal Form (3NF)

Each normal form addresses specific types of data anomalies and redundancy issues, leading to a more efficient and manageable database schema.

First Normal Form (1NF)

Explanation:

First Normal Form (1NF) requires that all attributes in a table be atomic, meaning each field must contain only a single value, and each record in the table must be unique and identifiable by a primary key. To meet 1NF, the table must have no repeating groups or arrays, with each column containing only one value per row.

Application of 1NF:

- ✓ The table must have no repeating groups or arrays. Each column should contain only one value per row.
- ✓ The primary key must uniquely identify each record, ensuring no duplicate rows.

Initial Table Structure:

BookingDataTable (VenueID, VenueNameID, VenueCategoryID, UserID, FirstName, LastName, Email, ContactNumber, PasswordHash, UserTypeID, EventTypeID, EventDescription, UserTypeName, UserTypeDescription, VenueCategoryName, VenueCategoryDescription, VenueName, VenueDescription, VenueCapacity, BookingID, StartDate, EndDate, StartTime, EndTime)

In Our Schema:

- **Booking Data Table**: The table is in 1NF as each column contains atomic values, and each row can be uniquely identified by the BookingID primary key. Attributes such as UserID, VenueID, BookingDate, BookingTime, DurationHours, and EventTypeID are all indivisible and fulfil the requirements for 1NF.
- **Note**: If there were a UserFullName attribute combining FirstName and LastName, we would need to split it into two separate attributes (FirstName and LastName) to meet 1NF requirements.

Steps Taken:

1. **Eliminated Repeating Groups**:	❖ Ensured no columns contain multiple values or repeating groups. ❖ Verified that all attributes are atomic
2. **Identified Primary Keys**:	❖ Defined primary keys for each table to ensure uniqueness and proper referencing: ➢ **Booking Table**: BookingID ➢ **User Table**: UserID ➢ **Venue Table**: VenueID ➢ **EventType Table**: EventTypeID ➢ **UserType Table**: UserTypeID ➢ **VenueName Table**: VenueNameID ➢ **VenueCategory Table**: VenueCategoryID

By adhering to these principles, we ensure that the data structure is organized efficiently, reduces redundancy, and meets the criteria for 1NF.

The following screenshots collectively represent a comprehensive view of the Booking Data Table. Due to the table's extensive size, it is captured across multiple images. Each screenshot displays a segment of the table, and together, these images provide a complete and detailed representation of the entire dataset.

Booking Data Table:

The Booking Data Table is effectively organized to adhere to 1NF by ensuring atomicity of attributes, maintaining a uniform domain for column values, using unique column names, and not relying on the order of data storage. This foundational normalization step establishes a robust framework for further normalization processes.

VenueID	VenueNameID	VenueCategoryID	UserID	FirstName	LastName	Email	ContactNumber	PasswordHash
V0000001	VN000001	VC0001	U000001	Alice	Smith	alice@example.com	123-456-7890	hash1
V0000002	VN000002	VC0002	U000002	Bob	Johnson	bob@example.com	234-567-8901	hash2
V0000003	VN000003	VC0003	U000003	Charlie	Brown	charlie@example.com	345-678-9012	hash3
V0000004	VN000004	VC0004	U000004	David	Wilson	david@example.com	456-789-0123	hash4
V0000005	VN000005	VC0001	U000005	Emily	Davis	emily@example.com	567-890-1234	hash5
V0000006	VN000006	VC0002	U000006	Frank	Miller	frank@example.com	678-901-2345	hash6
V0000007	VN000007	VC0003	U000007	Grace	Moore	grace@example.com	789-012-3456	hash7
V0000008	VN000008	VC0004	U000008	Henry	Thomas	henry@example.com	890-123-4567	hash8
V0000009	VN000009	VC0001	U000009	Isabella	Jackson	isabella@example.com	901-234-5678	hash9
V0000010	VN000010	VC0002	U000010	James	White	james@example.com	012-345-6789	hash10

UserTypeID	EventTypeID	EventDescription	UserTypeName	UserTypeDescription	VenueCategoryName	VenueCategoryDescription
UT0001	ET0001	Concert	Student	Regular student	Large Hall	Large capacity hall
UT0002	ET0002	Art Exhibition	Lecturer	Teaching staff	Small Gallery	Small gallery
UT0001	ET0003	Workshop	Student	Regular student	Medium Hall	Medium capacity hall
UT0002	ET0001	Concert	Lecturer	Teaching staff	Large Hall	Large capacity hall
UT0001	ET0004	Seminar	Student	Regular student	Small Hall	Small capacity hall
UT0002	ET0005	Meeting	Lecturer	Teaching staff	Medium Hall	Medium capacity hall
UT0001	ET0006	Training	Student	Regular student	Large Hall	Large capacity hall
UT0002	ET0002	Art Exhibition	Lecturer	Teaching staff	Small Gallery	Small gallery
UT0001	ET0003	Workshop	Student	Regular student	Medium Hall	Medium capacity hall
UT0002	ET0004	Seminar	Lecturer	Teaching staff	Large Hall	Large capacity hall

VenueName	VenueDescription	VenueCapacity	BookingID	StartDate	EndDate	StartTime	EndTime
Main Hall	Main venue	1000	B000001	2024/09/01	2024/09/02	18:00:00	23:00:00
Gallery	Art space	200	B000002	2024/10/01	2024/10/01	10:00:00	18:00:00
Conference Room	Meeting space	150	B000003	2024/11/01	2024/11/02	19:00:00	22:00:00
Main Hall	Main venue	500	B000004	2024/12/01	2024/12/02	20:00:00	01:00:00
Classroom	Class space	100	B000005	2024/12/15	2024/12/16	09:00:00	17:00:00
Conference Room	Meeting space	150	B000006	2024/11/15	2024/11/16	14:00:00	18:00:00
Main Hall	Main venue	500	B000007	2024/10/10	2024/10/11	08:00:00	16:00:00
Gallery	Art space	200	B000008	2024/09/20	2024/09/20	11:00:00	19:00:00

Second Normal Form (2NF)

2NF Requirements:

- ✓ The table must already be in First Normal Form (1NF).
- ✓ Remove partial dependencies, meaning non-key attributes must depend on the entire primary key, not just part of it.

Normalization Steps:

1. **Identify Partial Dependencies:** Partial dependencies occur when non-key attributes are dependent on only a part of a composite primary key, rather than the whole key. In our schema, we must ensure that attributes in a table depend on the entire primary key, not just a portion of it.

In the Booking Table:	✓ **VenueNameID** and **VenueCategoryID** are dependent only on **VenueID**. ❖ **Partial Dependency**: VenueID → VenueNameID ❖ **Partial Dependency**: VenueID → VenueCategoryID ✓ **UserTypeID** is dependent only on **UserID**. ❖ **Partial Dependency**: UserID → UserTypeID
In the Venue Table	✓ **VenueNameID** and **VenueCategoryID** are attributes that depend on **VenueID**. ❖ **Partial Dependency**: VenueID → VenueNameID ❖ **Partial Dependency**: VenueID → VenueCategoryID
In the User Table	✓ **UserTypeID** is an attribute that depends only on **UserID**. ✓ **Partial Dependency**: UserID → UserTypeID

2. **Make New Tables to Eliminate Partial Dependencies:** Here's how to address partial dependencies:
   - ✓ **Original Table with Partial Dependencies:** Let's consider a table with composite keys and partial dependencies. For instance, if a table has BookingID and VenueID as composite keys, and if attributes like VenueName depend only on VenueID (part of the composite key), this is a partial dependency.
   - ✓ **Example of Partial Dependencies:**
     - ❖ UserID → UserTypeID (in the User Table: UserTypeID depends only on UserID)
     - ❖ VenueID → VenueNameID, VenueCategoryID (in the Venue Table: VenueNameID and VenueCategoryID depend only on VenueID)

3. **Reassign Dependent Attributes to the Appropriate Tables:**

   To address partial dependencies, create new tables where attributes that depend only on part of the composite key are moved.

DECOMPOSITION

Initial Composite Table with Partial Dependencies: **Booking** (UserID, VenueID, BookingID, BookingDate, BookingTime, DurationHours, UserTypeID, VenueNameID, VenueCategoryID)
Normalized Tables: **Booking** (BookingID, UserID, VenueID, BookingDate, BookingTime, DurationHours) **User** (UserID, FirstName, LastName, ContactNumber, Email, PasswordHash, UserTypeID) **UserType** (UserTypeID, UserTypeName, UserTypeDescription) **Venue** (VenueID, VenueNameID, VenueCategoryID) **VenueName** (VenueNameID, VenueName, VenueDescription) **VenueCategory** (VenueCategoryID, VenueCategoryName, VenueCategoryDescription) **EventType** (EventTypeID, EventDescription)

Third Normal Form (3NF)

3NF Requirements:

- ✓ **The table must already be in Second Normal Form (2NF).**
- ✓ **Remove transitive dependencies, meaning non-key attributes must not depend on other non-key attributes.**

Normalization Steps:

1. **Identify Transitive Dependencies:** Transitive dependencies occur when a non-key attribute depends on another non-key attribute, rather than directly on the primary key. This can create redundancy and anomalies in data.

In the User Table:	**UserTypeID** → UserTypeName, UserTypeDescription (Here, UserTypeName and UserTypeDescription are dependent on UserTypeID, which is itself dependent on UserID.)
In the Venue Table:	**VenueNameID** → VenueName, VenueDescription (Here, VenueName and VenueDescription are dependent on VenueNameID, which is itself dependent on VenueID.) **VenueCategoryID** → VenueCategoryName, VenueCategoryDescription (Here, VenueCategoryName and VenueCategoryDescription are dependent on VenueCategoryID, which is itself dependent on VenueID.)
In the EventType Table:	**EventTypeID** → EventDescription (Here, EventDescription is dependent on EventTypeID, which is itself a foreign key in the Booking Table.)

1. **Make New Tables to Eliminate Transitive Dependencies:**

   Decompose tables to ensure all attributes are directly dependent on the primary key:

   User (UserID, FirstName, LastName, ContactNumber, Email, PasswordHash, UserTypeID)

   UserType (UserTypeID, UserTypeName, UserTypeDescription)

   Venue (VenueID, VenueNameID, VenueCategoryID)

   VenueName (VenueNameID, VenueName, VenueDescription)

VenueCategory (VenueCategoryID, VenueCategoryName, VenueCategoryDescription)

EventType (EventTypeID, EventDescription)

User Table	✓ **Primary Key:** UserID ✓ **Attributes:** UserID, FirstName, LastName, ContactNumber, Email, PasswordHash, UserTypeID ✓ **Dependencies:** Directly dependent on UserID.
UserType Table	✓ **Primary Key:** UserTypeID ✓ **Attributes:** UserTypeID, UserTypeName, UserTypeDescription ✓ **Dependencies:** UserTypeName, UserTypeDescription depend directly on UserTypeID.
Venue Table	✓ **Primary Key:** VenueID ✓ **Attributes:** VenueID, VenueNameID, VenueCategoryID ✓ **Dependencies:** Directly dependent on VenueID.
VenueName Table	✓ **Primary Key:** VenueNameID ✓ **Attributes:** VenueNameID, VenueName, VenueDescription ✓ **Dependencies:** VenueName, VenueDescription depend directly on VenueNameID.
VenueCategory Table	✓ **Primary Key:** VenueCategoryID ✓ **Attributes:** VenueCategoryID, VenueCategoryName, VenueCategoryDescription ✓ **Dependencies:** VenueCategoryName, VenueCategoryDescription depend directly on VenueCategoryID.
EventType Table	✓ **Primary Key:** EventTypeID ✓ **Attributes:** EventTypeID, EventDescription ✓ **Dependencies:** EventDescription depends directly on EventTypeID.
Booking Table	✓ **Primary Key:** BookingID ✓ **Attributes:** BookingID, UserID, VenueID, EventTypeID, BookingDate, BookingTime, DurationHours ✓ **Dependencies:** Directly dependent on BookingID. It has foreign keys to User, Venue, and EventType tables.

2. **Reassign Corresponding Dependent Attributes:**

   Move the attributes involved in transitive dependencies to their new tables:

- **User Table** now contains only attributes directly dependent on UserID.
- **UserType Table** contains attributes related to user types, removing transitive dependencies from the User table.
- **Venue Table** now contains only attributes directly dependent on VenueID.
- **VenueName Table** contains venue-related details.
- **VenueCategory Table** contains category-related details.
- **EventType Table** remains consistent with no transitive dependencies.

Normalized Tables:

User (UserID, FirstName, LastName, ContactNumber, Email, PasswordHash, UserTypeID)

UserType (UserTypeID, UserTypeName, UserTypeDescription)

Venue (VenueID, VenueNameID, VenueCategoryID)

VenueName (VenueNameID, VenueName, VenueDescription)

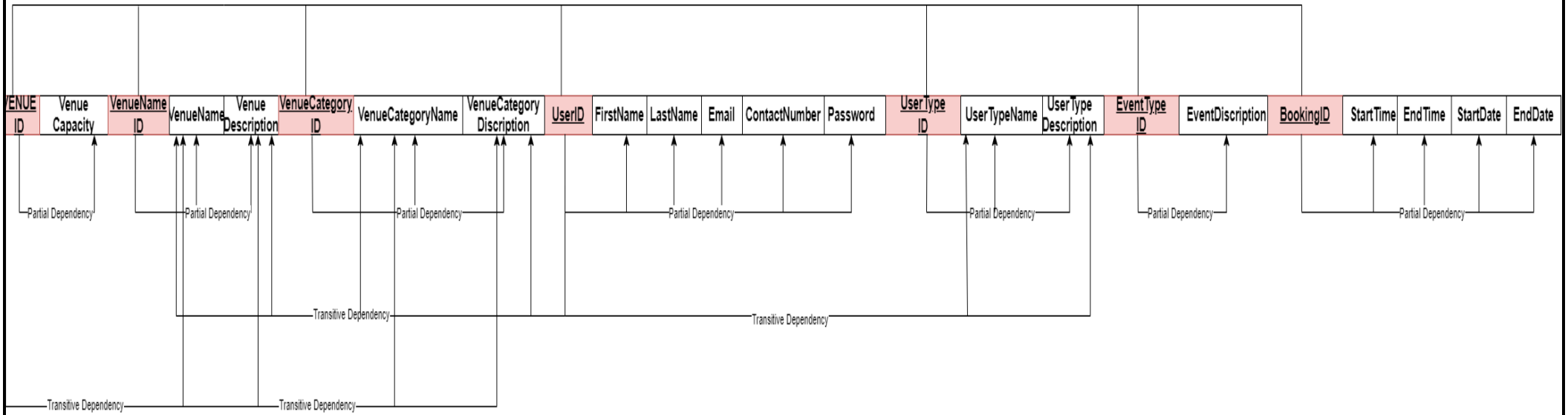
VenueCategory (VenueCategoryID, VenueCategoryName, VenueCategoryDescription)

EventType (EventTypeID, EventDescription)

Booking (BookingID, UserID, VenueID, BookingDate, BookingTime, DurationHours)

FULL DEPENDENCY DIAGRAM

To achieve normalization, it's essential to understand the relationships between attributes, primary keys, and foreign keys within our schema. Here's a summary of the dependencies and the full dependency diagram.



Primary Keys (PKs)	Partial Dependencies (PDs)	Transitive Dependencies (TDs)
✓ **User Table**: UserID ✓ **Venue Table**: VenueID ✓ **EventType Table**: EventTypeID ✓ **UserType Table**: UserTypeID ✓ **VenueName Table**: VenueNameID ✓ **VenueCategory Table**: VenueCategoryID ✓ **Booking Table**: BookingID	✓ **In the User Table**: UserID → UserTypeID ✓ **In the Venue Table**: VenueID → VenueNameID, VenueID → VenueCategoryID ✓ **In the Booking Table**: o UserID → FirstName, UserID → LastName, UserID → Email, UserID → ContactNumber, UserID → PasswordHash, UserID → UserTypeID o VenueID → VenueNameID, VenueID → VenueCategoryID o EventTypeID → EventDescription	✓ **In the UserType Table:** UserTypeID → UserTypeName, UserTypeID → UserTypeDescription ✓ **In the VenueName Table:** VenueNameID → VenueName, VenueNameID → VenueDescription ✓ **In the VenueCategory Table:** VenueCategoryID → VenueCategoryName, VenueCategoryID → VenueCategoryDescription ✓ **In the EventType Table:** EventTypeID → EventDescription

2. ENTITY RELATIONSHIP DIAGRAM (ERDs)

Introduction

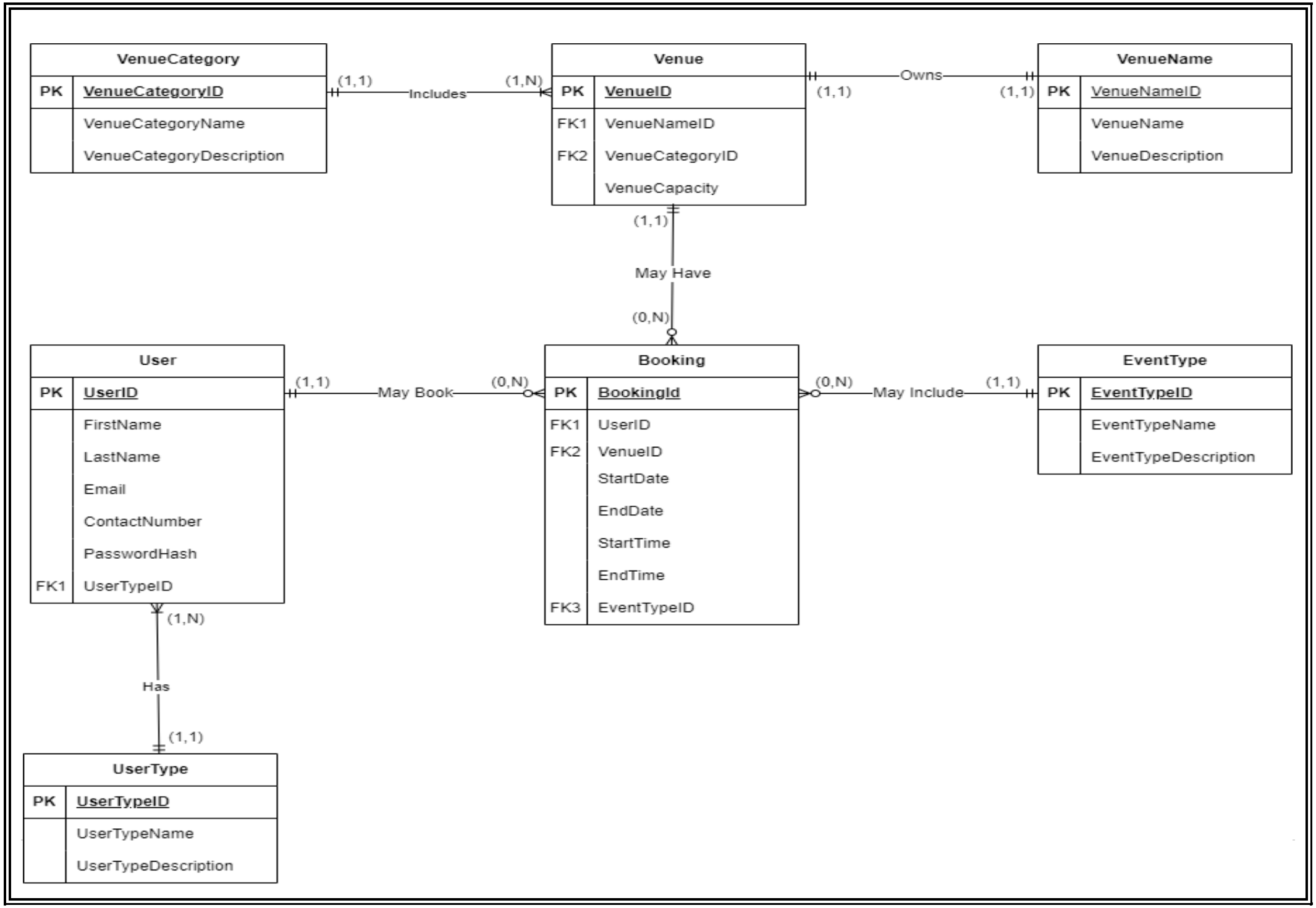
An **Entity-Relationship Diagram (ERD)** is a powerful tool used in database design to visually represent the entities within a database and the relationships between them. The primary function of an ERD is to map out the structure of a database, providing a clear and organized framework that depicts how data is interconnected.

Entities in an ERD represent distinct objects or concepts within the database that need to be stored. Each entity typically corresponds to a table in a relational database. For example, in a university database, entities might include Student, Course, and Instructor. Each entity is depicted as a rectangle, labelled with its name, and often includes a list of attributes that describe the entity. Attributes are the properties or characteristics of an entity, such as StudentID, StudentName, and EnrollmentDate for a Student entity.

Relationships illustrate how entities are related to one another. These relationships are depicted using lines connecting entities, often annotated with symbols or labels to specify the nature of the relationship. For instance, a Student entity might be connected to a Course entity through a relationship that represents enrolment, indicating that students can enrol in multiple courses and each course can have multiple students. This is often shown with a line connecting Student and Course, labelled with a verb phrase like "Enrols In."

Cardinalities describe the number of instances of one entity that can or must be associated with instances of another entity. Cardinality constraints specify whether the relationship is one-to-one, one-to-many, or many-to-many. For example, a one-to-many relationship between Instructor and Course indicates that an instructor can teach multiple courses, but each course is taught by only one instructor. This cardinality is typically denoted by symbols such as 1 for one, M for many, or 0.N for zero to many.

Attributes are the details associated with each entity, and these are crucial for defining the data structure. They can include primary keys (unique identifiers for each record) and other descriptive fields. For instance, the Student entity might include attributes like StudentID (the primary key), FirstName, LastName, and Email. Attributes are often represented as ovals connected to their respective entities in the ERD.
In summary, an ERD provides a comprehensive overview of the database design by visually representing entities, their attributes, and the relationships between entities. It serves as a blueprint for the database structure, aiding in the design, implementation, and maintenance of the database. By clearly defining entities, relationships, cardinalities, and attributes, an ERD ensures that the database is logically organized and that data relationships are well understood, facilitating efficient data management and integrity.


VenueCategory
PK VenueCategoryID
VenueCategoryName
VenueCategoryDescription
(1,1)
Includes
(1,N)
Venue
PK VenueID
FK1 VenueNameID
FK2 VenueCategoryID
VenueCapacity
(1,1)
Owns
(1,1)
VenueName
PK VenueNameID
VenueName
VenueDescription
(1,1)
May Have
(0,N)
User
PK UserID
FirstName
LastName
Email
ContactNumber
PasswordHash
FK1 UserTypeID
(1,1)
May Book
(0,N)
Booking
PK BookingId
FK1 UserID
FK2 VenueID
StartDate
EndDate
StartTime
EndTime
FK3 EventTypeID
(0,N)
May Include
(1,1)
EventType
PK EventTypeID
EventTypeName
EventTypeDescription
(1,N)
Has
(1,1)
UserType
PK UserTypeID
UserTypeName
UserTypeDescription

3. DOMAIN MODEL CLASS DIAGRAM

Introduction

Domain Model Class Diagram Overview

The Domain Model Class Diagram is a crucial component in designing a venue booking system, as it provides a detailed and organized representation of the system's core classes, their attributes, and the relationships between them. This diagram serves as a blueprint for understanding how different entities within the system interact and are structured.

In the context of our venue booking system, the Domain Model Class Diagram highlights the primary entities, such as User, Booking, Venue, EventType, VenueName, and VenueCategory. Each class is depicted with its attributes, which capture essential details about the entity, and methods, which define the operations that can be performed on or by the entity. The relationships between classes, such as one-to-many associations between User and Booking, and Venue and Booking, illustrate how different entities are interconnected. This includes relationships where one class contains or aggregates another, ensuring that data management and interactions are well-structured and efficiently handled.

Overall, the Domain Model Class Diagram not only matches the entities and attributes identified in the Entity-Relationship Diagram but also provides a comprehensive overview of the system’s architecture. It helps in visualizing how various parts of the system relate to each other, ensuring a coherent design and facilitating a clear understanding of the system’s functionality and data flow.

Classes in a Domain Model Class Diagram represent the fundamental building blocks of the system. Each class is depicted as a rectangle divided into compartments: the top compartment contains the class name, the middle compartment lists the class's attributes, and the bottom compartment (optional) includes methods or operations. For example, in a library management system, classes might include Book, Member, and Loan. Each class outlines the characteristics and behaviours pertinent to that entity.

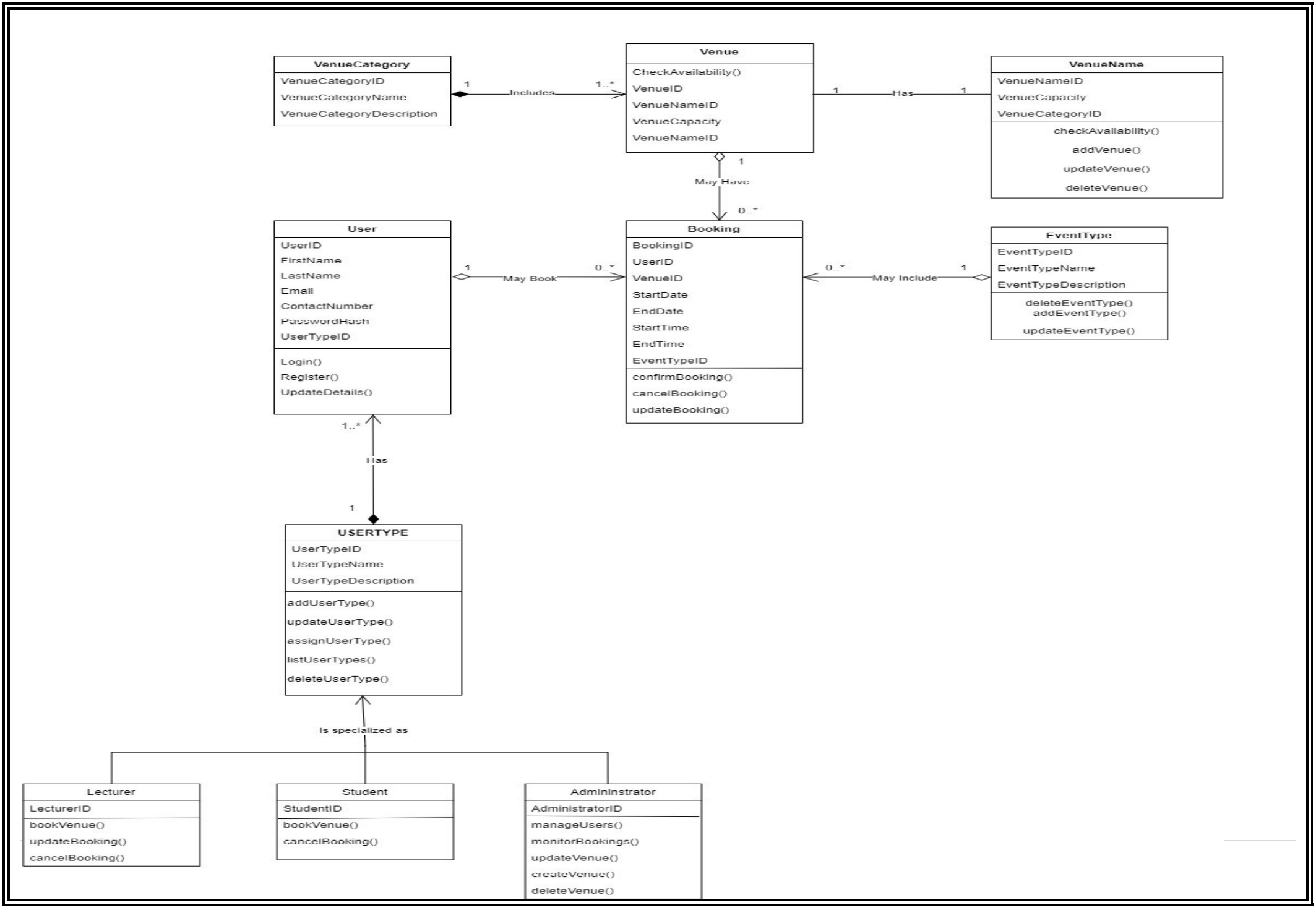
Attributes are the properties or data elements associated with a class. They describe the state or information held by the class instances. For example, the Book class might have attributes like ISBN, Title, Author, and Publication Year. These attributes define the data structure and are crucial for representing the properties of each object in the system.

Relationships between classes illustrate how different classes interact and are related. These relationships can include associations, generalizations (inheritance), and aggregations or compositions. For example, an Author class might have an association with the Book class indicating that an author can write multiple books, whereas a book can be authored by one or more authors. Relationships are shown using lines connecting classes, often with annotations to specify the nature of the relationship.

Associations indicate how classes are linked and can be annotated with multiplicities to specify the number of instances involved in the relationship. For instance, a Loan class might be associated with both Member and Book, showing that a member can borrow multiple books, and each book can be borrowed multiple times.

Generalization represents an inheritance relationship where a subclass inherits attributes and methods from a superclass. For instance, a PrintedBook class and an eBook class might both inherit from a common Book superclass, sharing common attributes and methods
Aggregation and **Composition** are specific types of associations that depict part-whole relationships. Aggregation represents a loose relationship where the part can exist independently of the whole (e.g., a Library and Book relationship), while composition denotes a stronger, dependent relationship where the part cannot exist without the whole (e.g., a House and Room relationship).

In summary, a Domain Model Class Diagram offers a comprehensive view of the system's structure, focusing on classes, their attributes, and their interrelationships. It facilitates a deep understanding of how objects within the system are organized and interact, serving as a foundational tool for both designing and documenting the system's architecture. By illustrating these elements, the diagram helps ensure that the system's design is coherent, maintainable, and aligned with its functional requirements.


VenueCategory
VenueCategoryID
VenueCategoryName
VenueCategoryDescription
1
Includes
1..*
Venue
CheckAvailability()
VenueID
VenueNameID
VenueCapacity
VenueNameID
1
Has
1
VenueName
VenueNameID
VenueCapacity
VenueCategoryID
checkAvailability()
addVenue()
updateVenue()
deleteVenue()
1
May Have
0..*
User
UserID
FirstName
LastName
Email
ContactNumber
PasswordHash
UserTypeID
Login()
Register()
UpdateDetails()
1
May Book
0..*
Booking
BookingID
UserID
VenueID
StartDate
EndDate
StartTime
EndTime
EventTypeID
confirmBooking()
cancelBooking()
updateBooking()
0..*
May Include
1
EventType
EventTypeID
EventTypeName
EventTypeDescription
deleteEventType()
addEventType()
updateEventType()
1..*
Has
1
USERTYPE
UserTypeID
UserTypeName
UserTypeDescription
addUserType()
updateUserType()
assignUserType()
listUserTypes()
deleteUserType()
Is specialized as
Lecturer
LecturerID
bookVenue()
updateBooking()
cancelBooking()
Student
StudentID
bookVenue()
cancelBooking()
Admininstrator
AdministratorID
manageUsers()
monitorBookings()
updateVenue()
createVenue()
deleteVenue()

4. TEAM SIGN-OFF

INFO2001A - PEER EVALUATION

Team No: __32__ Team Name: _____TECHFUSION_____ Date: ___23/08/2024___

Each team is required to complete a peer evaluation. The following aspects of the team's work should be taken into account when deciding on the percentage contribution to allocate to each member:

- Quality of individual contribution
- Timely submission of work
- Online Availability and communication

The following principles apply:

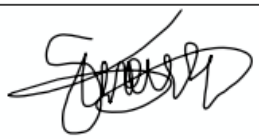
1. Every team member must agree on percentage contributions and sign the form.
2. The evaluation is not anonymous, but an adult assessment of contribution.
3. Think carefully about your assessment of your peers. Try to avoid extreme assessments unless they are fully justified.
4. In a team of five (5), if each member contributed equally, then each would be allocated 20%, in a team of six (6) each would be allocated 16.67%.
5. A team member who scores higher than the average will be awarded a project mark that is proportionally above the mark obtained by the team.
6. A team member who scores lower than the average will be awarded a project mark that is proportionally below the mark obtained by the team.
7. Extreme cases, e.g. where a student has made no contribution at all, will be handled on a case by case basis by the project coordinator (lecturer)

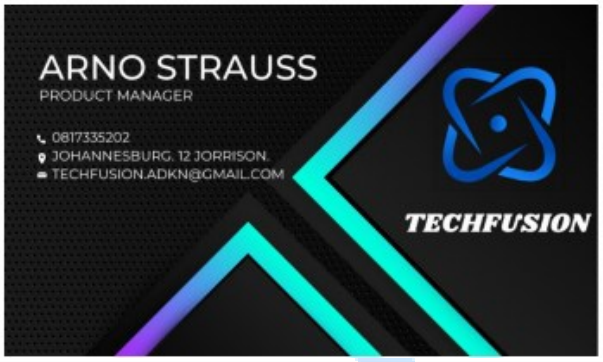
Member	Member Name	Contribution (%)
1	Arno Strauss	33.34%
2	Dintle Maine	33.34%
3	Khulekani Mtshali	33.34%
4		
5		
6		
Total		**100%**

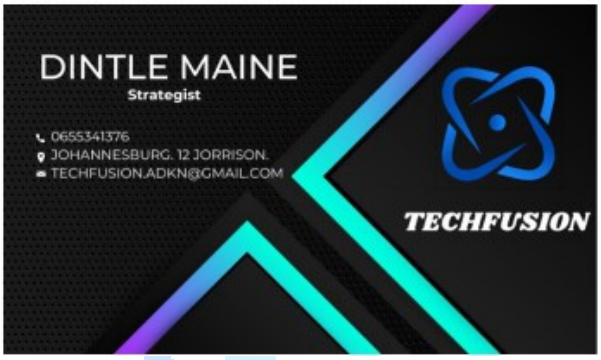
Team Signatures

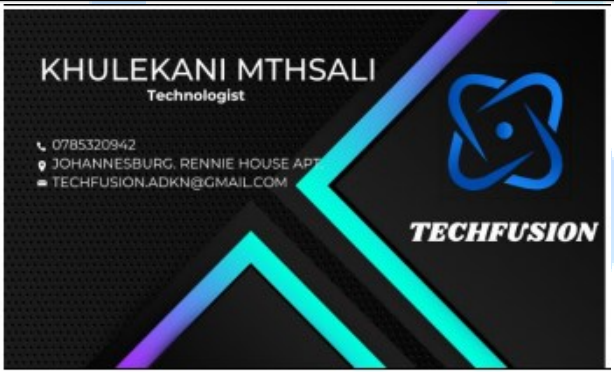
Member	Member Signatures
1	A. Strauss
2	D. Maine
3	K. Mtshali
4	
5	
6	

Motivation (where there are issues that need explanation):

Arno Strauss	
Dintle Maine	
Khulekani Mtshali	







WITS UNIVERSITY

SCHOOL OF BUSINESS SCIENCES

RESERVIFY® Design Phase

Milestone 1: Revised Use Case Set + CRUD table

Group Number: 32

Team Name: TechFusion

TechFusion Partner Details

Member:	Name:	Surname:	Student Number:
1	Arno	Strauss	2613224
2	Dintle	Maine	2451837
3	Khulekani	Mtshali	2550411

TechFusion Logo

TABLE OF CONTENTS

1. CRUD Tabular Structure

ENTITY	VERIFIED USE CASES	RELATED USE CASES
STUDENT	Create Student Profile	
	View Student Profile	
	Update Student Profile	View Student Profile(includes)
	Delete Student Profile	View Student Profile(includes)
	Create Booking	View Venue(includes)
	View Booking	
	Delete Booking	View Booking(includes)
LECTURER	Create Lecturer Profile	
	View Lecturer Profile	
	Update Lecturer Profile	View Lecturer Profile(includes)
	Delete Student Profile	View Student Profile(includes)
	Create Booking	View Venue(includes)
	View Booking	
	Update Booking	View Booking(includes)
	Delete Booking	View Booking(includes)
ADMINISTRATOR	View Administrator/Student/Lecturer Profile	
	Update Administrator/Student/Lecturer Profile	View Administrator/Student/Lecturer Profile(includes)
	Delete Student/Lecturer Profile	View Student/Lecturer(includes)
	Create Venue	
	View Venue	
	Update Venue	View Venue(includes)
	Delete Venue	View Venue(includes)
	Generate Reports	
VENUE	Create Venue	
	View Venue	
	Update Venue	View Venue(includes)
	Delete Venue	View Venue(includes)
BOOKING	Create Booking	View Venue(includes)
	View Booking	
	Update Booking	View Booking(includes)
	Delete Booking	View Booking(includes)

2. REVISED USE CASE SET

Use Case Name:	Create Profile
Use Case Description:	This use case details the process for a student to **create a new profile** on the **venue booking system**, Reservify. The student initiates the **profile creation** by selecting the "**Register**" option on the registration page of Reservify. The registration form requires the student to input their **personal information:** first name, last name, email address, and contact number. Additionally, the student must select their **user type** from available options (e.g., "Student" or "Lecturer") this use case activates when the student selects the user type “Student”. Upon submission of the **registration form**, the system performs a series of **validation checks** to ensure the accuracy and completeness of the provided information. These checks include verifying the format of the email address, ensuring that no fields are left blank, and confirming that the email address is unique in the system. If all validations are **successful**, the system securely **stores** the student's information in the database. The system generates a unique profile identifier for the student and creates a new record in the student profile database table. Following successful profile creation, the system displays a **confirmation message** to the student, indicating that their profile has been successfully created. The **student** is then redirected to the **login page** or the **home page** of the system. On the login page, the student can enter their credentials to access the system’s features, including **venue booking functionalities**. If the registration process encounters any errors or validation issues, the student is redirected back to the **registration page** with detailed error messages indicating what needs to be corrected. The system ensures that no partial or incorrect data is saved, and the registration attempt does not create an incomplete profile.
Actor(s):	**Student**
Pre-Conditions:	**PR1**. The student has **accessed** the venue booking system, Reservify. **PR2.** The student has **navigated** to the fully operational and error-free **registration page**. **PR3:** The student must not have an **existing profile** in the Reservify system; (i.e. their email address or contact number) must be unique. **PR4:** The Reservify system must be **online and functional** at the time of registration.
Post-Conditions:	**Success:** **PO1**: The system securely saves the student's **personal information** in the database. The data includes the student’s first name, last name, email address, contact number, and user type. The **student profile** is created and **stored** in the system with a unique identifier. **PO2**: The system generates and **displays a confirmation message** to the student indicating successful profile creation. **PO3**: The student is **redirected** to the login page or **home page**, allowing them to log in and access booking features.

	Failure: **PO4**: If **registration fails** due to invalid input or system errors, the student is redirected to the **home page**. **PO5**: The system does not save any partial or **incomplete data**, and no profile is created.	
Main Flow:	**Actor**	**System**
	S1. The student **launches** the Reservify application or accesses the desktop interface and clicks on the "**Register**" option.	S1.1. Displays the **registration interface**, which includes fields for the student's **personal information**, user type selection, and account setup. The form includes fields for entering the first name, last name, email address, contact number, and user type. There is also a section to create a password and confirm it.
	S2. The **student enters their personal information** (the first name, last name, email address, contact number) into the registration form, selects “Student” as their user type, and creates a password.	**S2.1** The system dynamically **validates the input fields**, providing immediate feedback (e.g., ensuring passwords match, highlighting required fields). The system prompts the user to confirm the accuracy of their information.
	S3. The student **reviews the entered information** for accuracy and completeness, then clicks the “Submit” button to finalize the registration.	S3.1. Performs backend **validation** of the submitted information: Ensures all **required fields** (first name, last name, email, contact number, user type, and password) **are filled out**. Verifies the format of the email address (e.g. correct syntax) and contact number (e.g., proper format). Checks the email address and contact number for uniqueness in the database to prevent duplicate profiles. S3.2. **If validation is successful**: S3.2.1 **Creates** a new student profile in the database with a unique identifier. S3.2.2 **Stores** the student's personal information, including the chosen user type and hashed password. S3.2.3 **Generates** a confirmation message indicating that the profile has been successfully created. S3.3. **Updates** the user interface to show that the profile creation was successful with **confirmation message** and offers options to proceed with login or other actions available on the **home page**. S3.4. **Redirects** the student to the login page or the **home screen** of the application, where the student can log in using their **new credentials**.

Use Case Name:	Update Venue Booking
Use Case Description:	The "**Update Venue Booking"** use case enables lecturers to modify the details of their existing venue bookings within the **Reservify system**. This process begins when a lecturer accesses the system and navigates to the "Update Venue Booking" page. The lecturer **selects** the specific booking they wish to modify from a list or through a search function. Upon selecting the booking, the system **displays** the current details in an editable form, including the type of event, category of venue, capacity, dates, and duration. The lecturer **reviews and updates** these details as necessary. The changes may involve adjusting the event type (e.g., changing from a "Society Meeting" to a "Conference"), selecting a different venue category (e.g., from "Lecture Room" to "Conference Room"), modifying the capacity to accommodate more attendees, or updating the dates and duration of the booking. This ensures that the booking reflects the lecturer's current needs and preferences. After the lecturer submits the updated information, the system performs a **series of validation checks** to ensure that the new details are correct and adhere to the system's rules. Validation includes **verifying that the new event type is valid and available**, ensuring the selected venue category is appropriate for the event, checking that the requested capacity **does not exceed** the venue's maximum limit, confirming that the new dates do not overlap with existing bookings, and ensuring the duration is within acceptable time frames. If the system **detects any errors** or conflicts during these checks, it provides specific error messages to guide the lecturer in correcting the issues. The system prevents the submission of invalid updates, ensuring the integrity and accuracy of the booking information. Upon **successful validation**, the system updates the booking record in the database with the new details. The lecturer receives a **confirmation message** indicating that the booking has been successfully updated. This confirmation includes the revised booking details, ensuring that the lecturer can verify that the changes have been applied correctly. The **updated booking information** is then reflected in the lecturer's booking history or booking summary page. **In cases of failure**, such as system errors or validation issues, the lecturer is informed of the problems and prompted to take corrective actions. This comprehensive approach ensures that the booking update process is accurate, user-friendly, and reliable, accommodating the lecturer's needs while maintaining the system's data integrity.
Actor(s):	**Lecturer**
Pre-Conditions:	**PR1**: The venue booking system, Reservify, must be **fully operational** and accessible without any system outages or errors. **PR2**: The lecturer must be **authenticated**, having logged into the Reservify system with appropriate Lecturer Rights or permissions to update bookings. **PR3**: The lecturer must be on the "**Update Venue Booking**" page or form within the system. **PR4**: The lecturer must have an **existing booking** that they intend to update. The booking must be identifiable within the system (e.g., through a booking ID or reference number).
Post-Conditions:	**Success:** **PO1**: The venue booking system **successfully updates** the booking record in the database with the new details provided by the lecturer. The updated information includes changes to the type of event, category of venue, capacity, dates, and duration.

<table>
<tr><td></td><td colspan="2">PO2: The system displays a confirmation message to the lecturer, indicating that the booking has been successfully updated. The updated booking details are accurately reflected in the system.

PO3: The lecturer can view the updated booking details in their booking history or booking summary page.

<u>Failure:</u>
PO4: If the system encounters an error while processing the update (e.g., due to validation failures or system issues), the booking remains unchanged, and the system displays an error message. The lecturer is informed that the update was unsuccessful, and they may need to correct the information and try again.

PO5: If the update fails due to invalid input (e.g., conflicting dates, incorrect capacity (overcapacity booking), the system provides specific error messages detailing the issues. The lecturer is prompted to correct the information and resubmit the update request.</td></tr>
<tr><td rowspan="4">Main Flow:</td><td>Actor</td><td>System</td></tr>
<tr><td>S1. The lecturer navigates to the "Update Venue Booking" page.</td><td>S1.1 The system displays a list of the lecturer's current bookings.</td></tr>
<tr><td>S2. The lecturer selects the booking they wish to update.</td><td>S2.1 The system shows the selected venue booking details in an editable format.</td></tr>
<tr><td>S3. The lecturer modifies the booking details (e.g., event type, venue category, capacity, dates, duration).</td><td>S3.1 The system provides real-time validation and feedback, ensuring inputs are correct and conflicts are identified immediately (Also a background backup is kept for when the Lecturer goes offline unexpectedly)</td></tr>
<tr><td>S4. Lecturer clicks the button to submit the updated booking information, finalizing the changes and ensuring the new details are recorded.</td><td>S4.1 System validates the updated information:
• Ensures all required fields are updated correctly.
• Verifies the new booking details for availability.
• Checks for any conflicts with existing bookings.

S4.2 If validation passes:
• System updates the booking details in the Reservify database.
• A confirmation email with the updated booking details is sent to the lecturer.
• The system generates a success confirmation message.

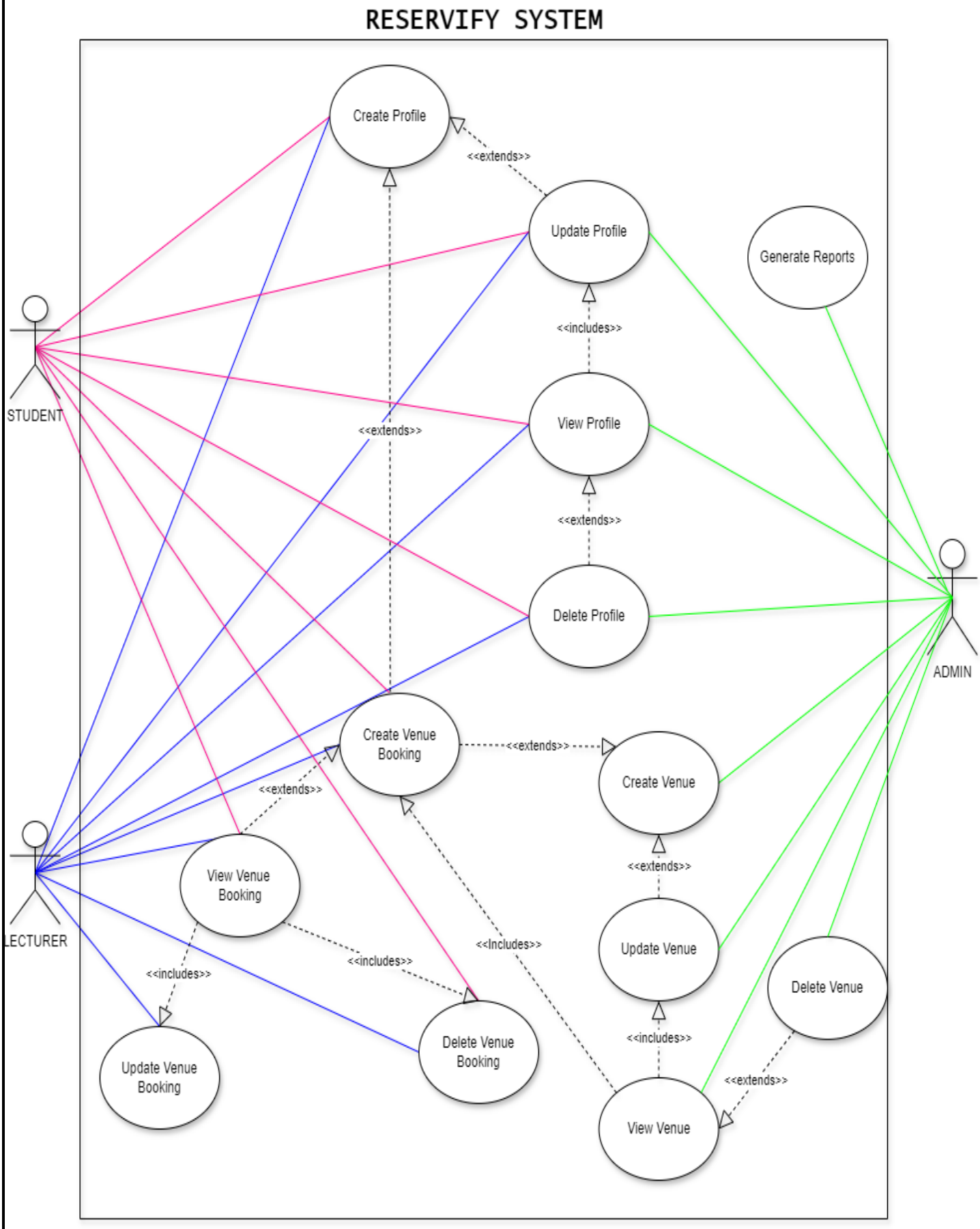
S4.3 System displays a confirmation message indicating successful booking update.

S4.4 The updated booking details are reflected in the lecturer's booking history or summary page.</td></tr>
</table>

Use Case Name:	Update Student
Use Case Description:	The "**Update Student**" use case allows an administrator to **modify** the details of a student's profile within the Reservify system. The process starts when the administrator **logs** into the system with the appropriate credentials, granting them administrator rights. Once logged in, the administrator navigates to the "**Update Profile**" page via the system's dashboard. On this page, the administrator utilizes a search or selection interface to locate the specific student whose profile requires updating. This **search** functionality allows the administrator to input identifiers such as the student's unique ID, name, or email address to retrieve the student's current profile from the database. Upon **locating the student's profile**, the system displays the **current details**, including the student's first name, last name, email address, and contact number, in an editable format. The **administrator reviews** these details and makes the necessary changes. For instance, if a student has recently changed their contact number, the administrator **updates** the contact number field. The system incorporates real-time **validation checks** as the administrator enters the new information. These checks ensure that the email address follows a **valid format** (e.g., contains an "@" symbol and a valid domain), that the contact number is composed of numerical digits only, and that any changes do not create inconsistencies, such as duplicate email addresses. After making the changes, the administrator submits the **updated information**. The system processes this update, performing additional **validation** to confirm that all new data adheres to system rules and constraints. If the validation is **successful**, the system updates the student's profile in the database and generates a **confirmation message** for the administrator, indicating that the update was completed successfully. The system also sends a to the student via email or through the system's internal notification platform, informing them of the updated details and providing a summary of the changes made. In case of an update failure due to system errors, validation issues, or other reasons, the system **notification** prevents the changes from being saved and provides **error messages** to the administrator to correct the issues. The student's profile remains unchanged, and no notification is sent to the student regarding the **failed update attempt**.
Actor(s):	Administrator
Pre-Conditions:	**PR1**. The venue booking system Reservify **is fully operational** and accessible to **authorized** users. **PR2**. The administrator has **successfully logged** into Reservify with administrator rights, ensuring proper authentication. **PR3**. The administrator is currently on the "**Update Profile**" page/form within the system. **PR4**. The student profile that the administrator intends to update **exists** in the Reservify database and is **accessible** for modification.
Post-Conditions:	**Success:** **PO1**. The student's profile is **successfully updated** with the new details provided by the administrator, and these changes are accurately reflected in the Reservify database. **PO2**. The administrator receives a **confirmation message** indicating the successful update of the student's profile. Additionally, the student receives a notification that their profile details have been updated, along with the updated information for their review. **Failure:**

	PO3. The student profile update process **fails** due to issues such as **validation errors** or system malfunctions, and the profile remains **unchanged** in the database. **PO4**. In the event of a **failure**, the system does not notify the administrator or the student about the failure, and the update attempt is **not saved**.	
Main Flow:	**Actor**	**System**
	S1. The administrator **navigates** to the "Update Profile" page from the dashboard.	**S1.1** The system **displays** the "**Update Profile**" page with a search or selection interface to locate student profiles.
	S2. The administrator **searches** for the student profile using identifiers such as student ID, name, or email address.	**S2.1** The system **processes** the search query, retrieves, and displays a list of student profiles that match the search criteria
	S3. Administrator **selects** the student profile they want to update/modify/edit from the list.	**S3.1** The system **fetches and displays** the current details of the selected student profile in an editable form. The form includes fields for the student's first name, last name, email address, contact number, and user type.
	S4. The administrator **updates** the necessary student information, such as name, email, contact number, and user type.	**S4.1** The system performs **real-time validation** on the updated fields, ensuring that the email format is correct, the contact number is numerical, and there are no duplicate entries.
	S5. The administrator **reviews all the updated information** for accuracy and completeness, then submits the changes by clicking the "**Update**" button**.**	S5.1 System **validates** the updated student information: • Confirms all required fields are filled correctly. • Verifies the format of the entered data (e.g. email, phone number). **S5.2 If validation is successful**: **S5.2.1** The system **updates** the student profile in the database with the new details. **S5.2.2** The **system logs the update action** for audit purposes, including the administrator's ID and timestamp of the update. **S5.2.3** The system sends an **email notification** to the student, informing them of the changes to their profile with a summary of the updated details. **S5.2.4** The system **generates a confirmation message**, indicating the successful update of the student profile. **S5.3** System **displays** the confirmation message to notify the administrator of the successful update of the student profile. **S5.4** The system **redirects** the administrator back to the student **profile management page**, showing the updated student details and providing options for further actions or to return to the dashboard.

3. REVISED USE CASE DIAGRAM



4. TEAM SIGN-OFF

INFO2001A - PEER EVALUATION

Team No: __32__ Team Name: _____TECHFUSION_____ Date: ___31/07/2024___

Each team is required to complete a peer evaluation. The following aspects of the team's work should be taken into account when deciding on the percentage contribution to allocate to each member:

- Quality of individual contribution
- Timely submission of work
- Online Availability and communication

The following principles apply:

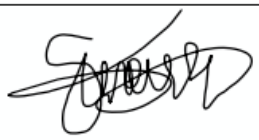
1. Every team member must agree on percentage contributions and sign the form.
2. The evaluation is not anonymous, but an adult assessment of contribution.
3. Think carefully about your assessment of your peers. Try to avoid extreme assessments unless they are fully justified.
4. In a team of five (5), if each member contributed equally, then each would be allocated 20%, in a team of six (6) each would be allocated 16.67%.
5. A team member who scores higher than the average will be awarded a project mark that is proportionally above the mark obtained by the team.
6. A team member who scores lower than the average will be awarded a project mark that is proportionally below the mark obtained by the team.
7. Extreme cases, e.g. where a student has made no contribution at all, will be handled on a case by case basis by the project coordinator (lecturer)

Member	Member Name	Contribution (%)
1	Arno Strauss	33.34%
2	Dintle Maine	33.34%
3	Khulekani Mtshali	33.34%
4		
5		
6		
Total		**100%**

Team Signatures

Member	Member Signatures
1	A. Strauss
2	D. Maine
3	K. Mtshali
4	
5	
6	

Motivation (where there are issues that need explanation):

Arno Strauss	
Dintle Maine	
Khulekani Mtshali